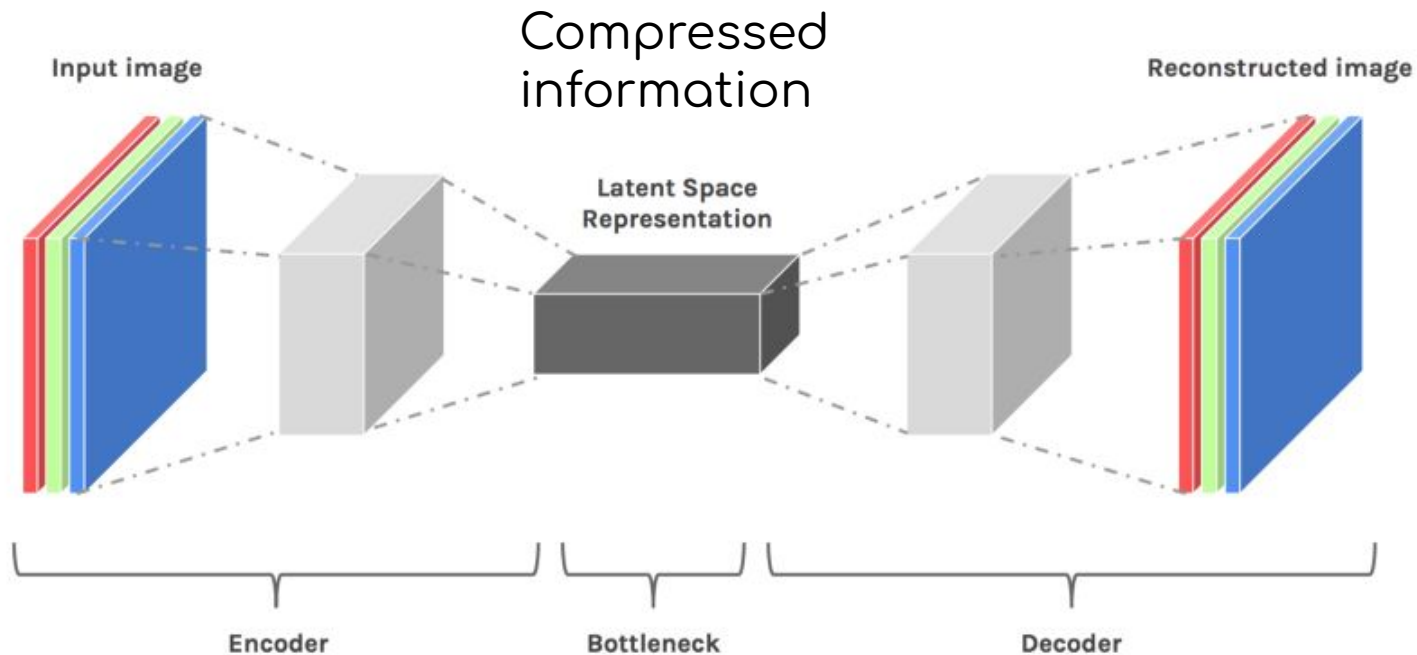


Computer vision

Object detection using Deep Neural
Networks (DNN)

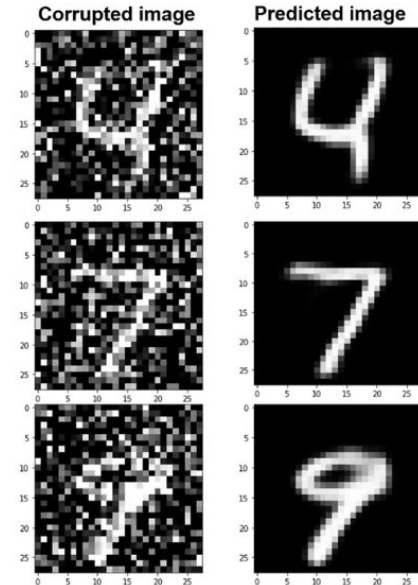
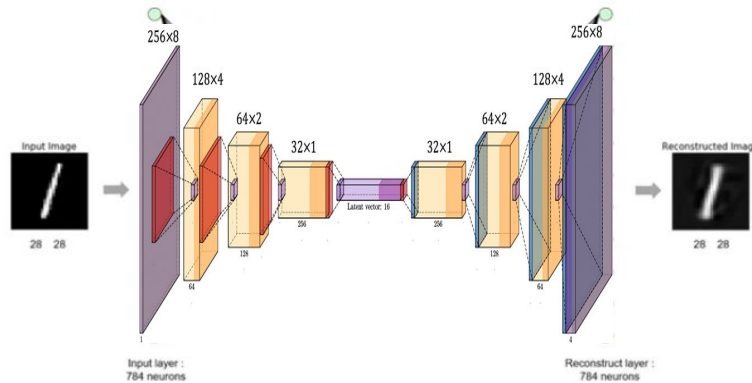
Autoencoder

CNN Autoencoder



Example of using CNN: Noise reduction using CNN

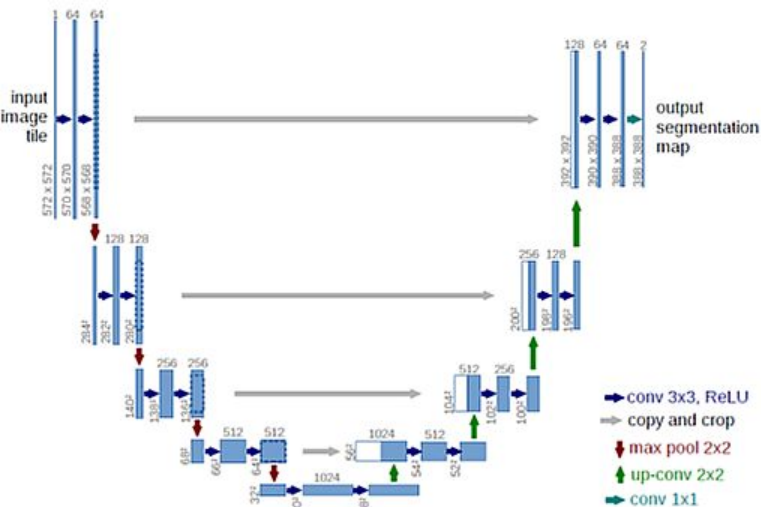
Denoising CNN Autoencoder



U-net

U-Net architecture

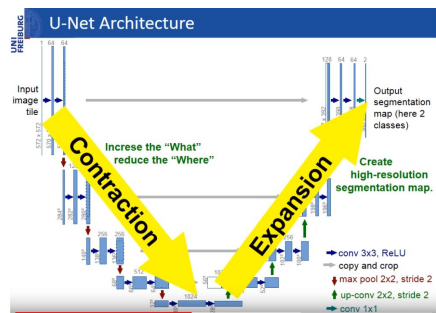
example for 32x32 pixels in the lowest resolution:



- Each blue box corresponds to a multi-channel feature map.
- The number of channels is denoted on top of the box.
- The x-y-size is provided at the lower left edge of the box.
- White boxes represent copied feature maps.
- The arrows denote the different operations.

U-Net architecture

It consists of a contracting path
(left side)
and an expansive path
(right side).



contracting path (left side)

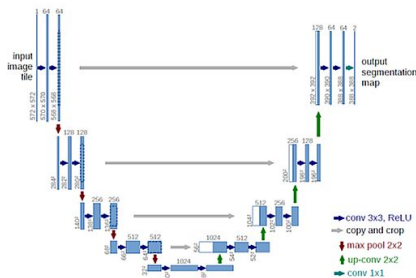
Follows the typical architecture of a convolutional network:

- It consists of the repeated application of two 3x3 convolutions (unpadded convolutions), each followed by a rectified linear unit (ReLU) and a 2x2 max pooling operation with stride 2 for downsampling.
- At each downsampling step we double the number of feature channels.

expansive path (right side)

Every step in the expansive path consists of

- upsampling of the feature map followed by a 2x2 convolution ("up-convolution") that halves the number of feature channels,
- a concatenation with the correspondingly cropped feature map from the contracting path,
- and two 3x3 convolutions, each followed by a ReLU. The cropping is necessary due to the loss of border pixels in every convolution.
- At the final layer a 1x1 convolution is used to map each 64- component feature vector to the desired number of classes.



2D Transposed Convolution

„Deconvolution block“ used in Expansion

The 2D “Deconvolution block” represents a layer that performs an **opposite operation** to 2D Convolution.

The term „Deconvolution“ is not exact – NOT a Deconvolution in terms of signal theory

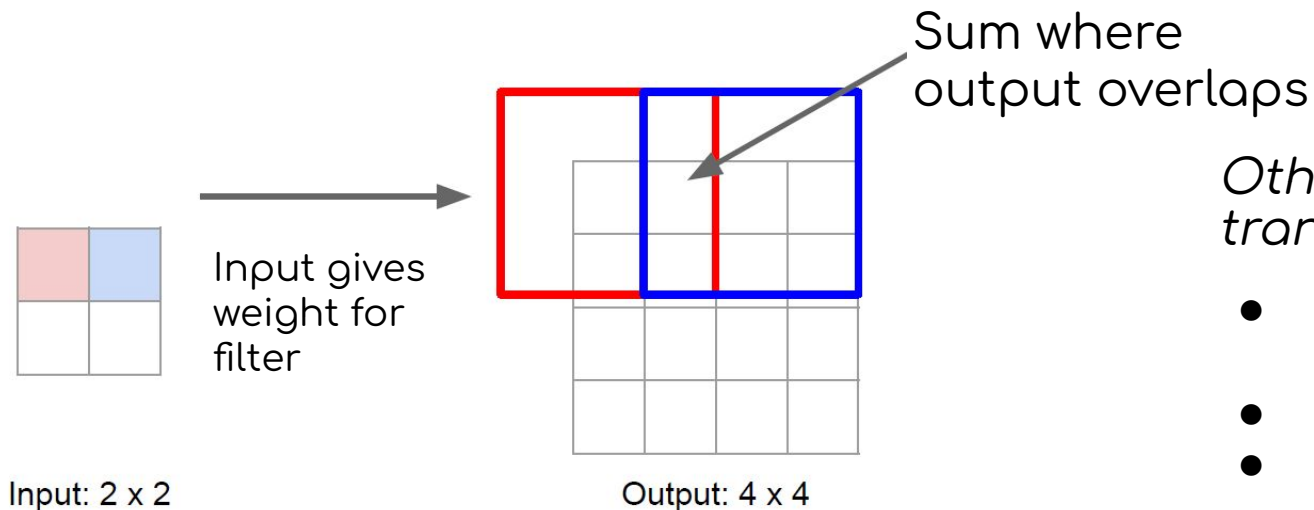
- Deconvolution blocks **have to learn weights** in the same way as Convolution layers
- operate similarly to convolution blocks, **but stride has the opposite effect** (making the output bigger, not smaller).

Usage: The deconvolution operation is used when we transform the output of a convolution back into a tensor that has the same shape as the input of that convolution.

Autoencoders and image segmentation models (U net) are examples of such models.

Upsampling: Transpose Convolution

3 x 3 transpose convolution, stride 2 pad 1

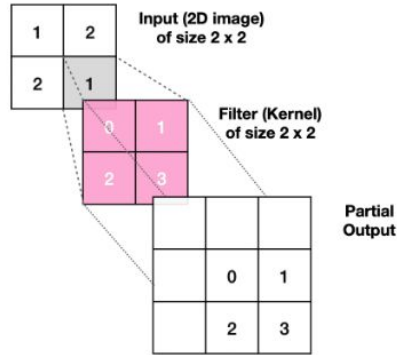


Other names of transpose convolution:

- Deconvolution (not exact)
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution

2D Transposed Convolution - example

Transposed
convolution layer
with a 2×2 kernel.



0	1	
2	3	

 +

	0	2
	4	6

 +

0	2	
4	6	

 +

	0	1
	2	3

 =

=

0	1	2
2	9	7
4	8	3

Final Output

U-Net Training

The input images and their corresponding segmentation maps are used to train the network with the stochastic gradient descent (SGD).

Commonly used loss functions:

- Binary cross-entropy loss

$$L = -[y \cdot \log(\rho) + (1-y) \cdot \log(1-\rho)]$$

Where: y is the true label (0 or 1)

ρ is the predicted probability

- penalizes confident but incorrect predictions
- logarithmic function.

If the true label is 1, a low predicted probability results in a large penalty.
If the true label is 0, a high predicted probability results in a large penalty.

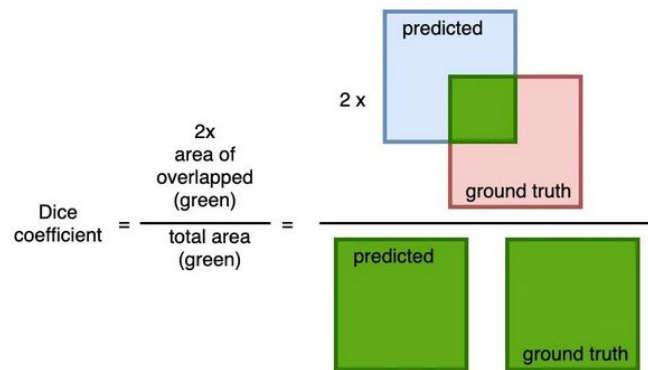
U-Net Training

Commonly used loss functions:

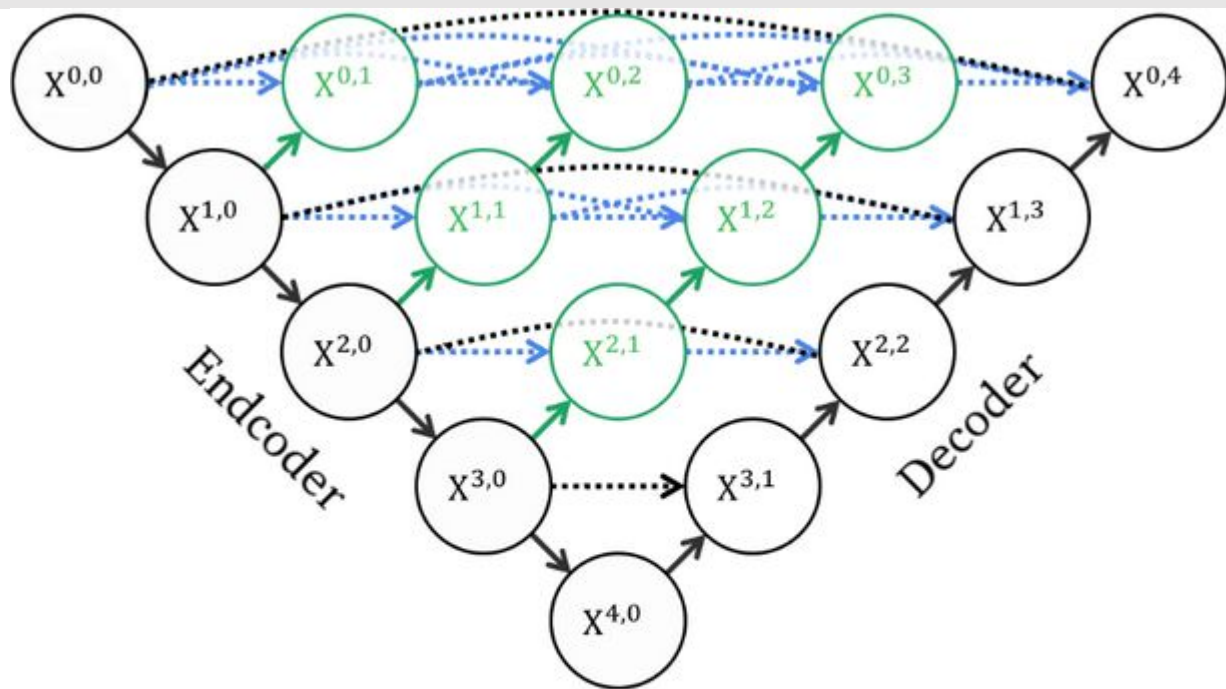
- Dice loss:

$$L = 1 - DICE$$

$$DICE = \frac{2 |S_g^1 \cap S_t^1|}{|S_g^1| + |S_t^1|} = \frac{2TP}{2TP + FP + FN}$$



U-Net ++



$$x^{0,1} = H[x^{0,0}, U(x^{1,0})], \quad x^{0,2} = H[x^{0,0}, x^{0,1}, U(x^{1,1})], \quad \dots$$

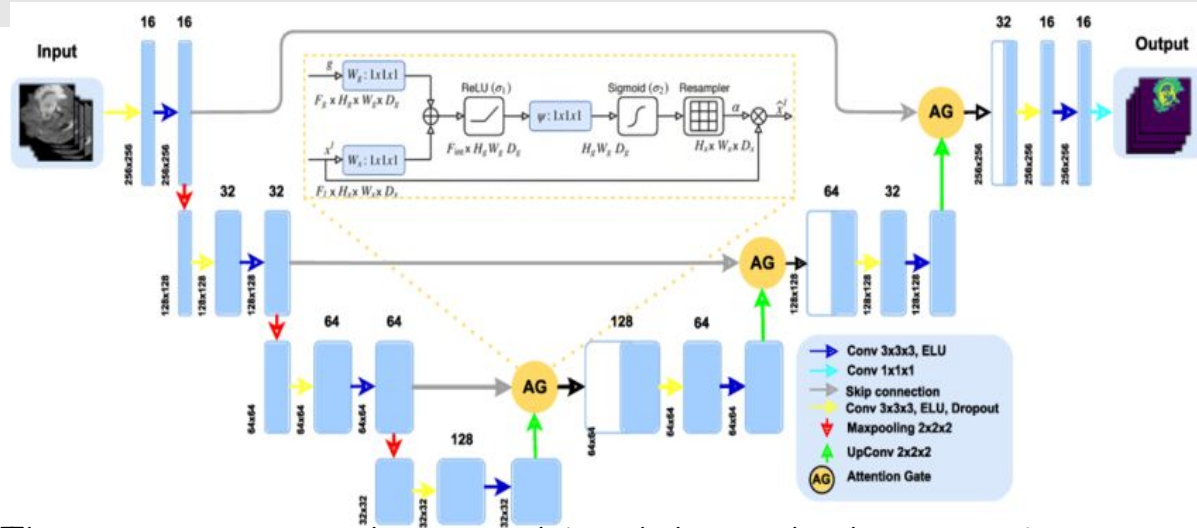


Down-sampling (2*2)



Skip connection

Attention U-net



The attention gate AG involves two input vectors: X and G.

Vector G is derived from the lower layer of the network and has smaller dimensions but better feature representation.

Vector X undergoes a strided convolution, while vector G undergoes a 1x1 convolution.

These vectors are then combined through element-wise summation, resulting in a new vector. This new vector is passed through activation and convolutional layers to reduce its dimensions. A sigmoid layer is then applied to produce attention coefficients, which represent the importance of each element in vector X.

To restore the attention coefficients to the original dimensions, trilinear interpolation is used. The attention coefficients are multiplied with the original vector X, scaling it based on relevance. Finally, the modified vector is passed through the skip connection for further processing.

Object recognition using CNN

Image Classification: A core task in Computer Vision



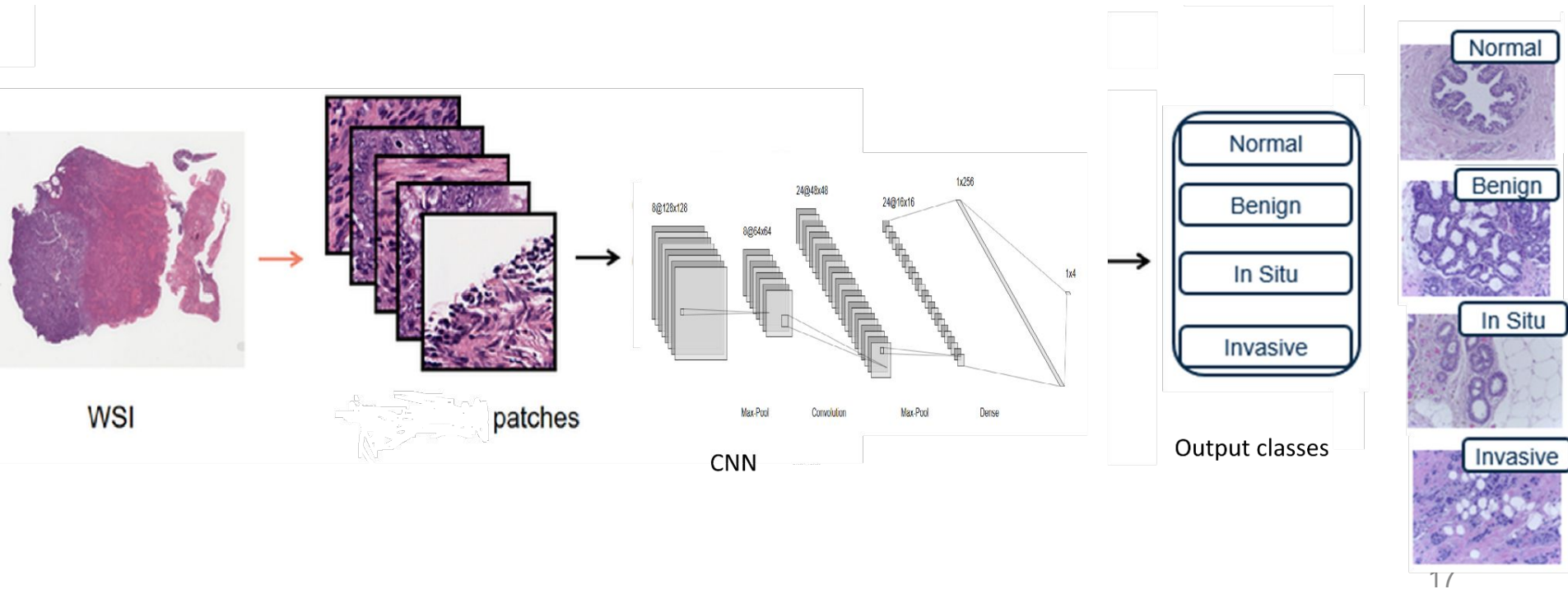
assume given a set of possible labels:
dog, cat, truck, plane, ...



cat

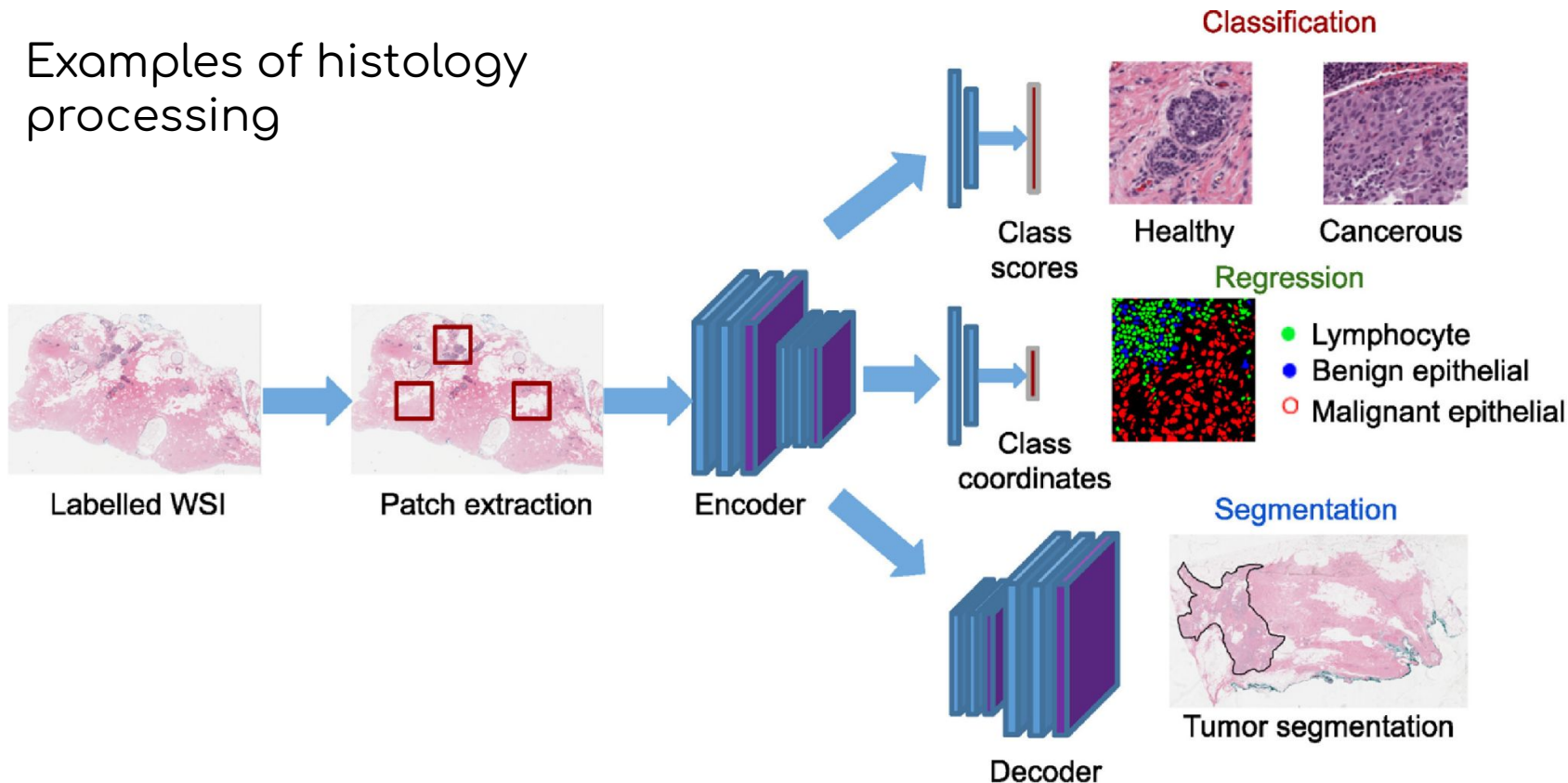
CNN - example of classification task in medical domain

Classification of histology patches

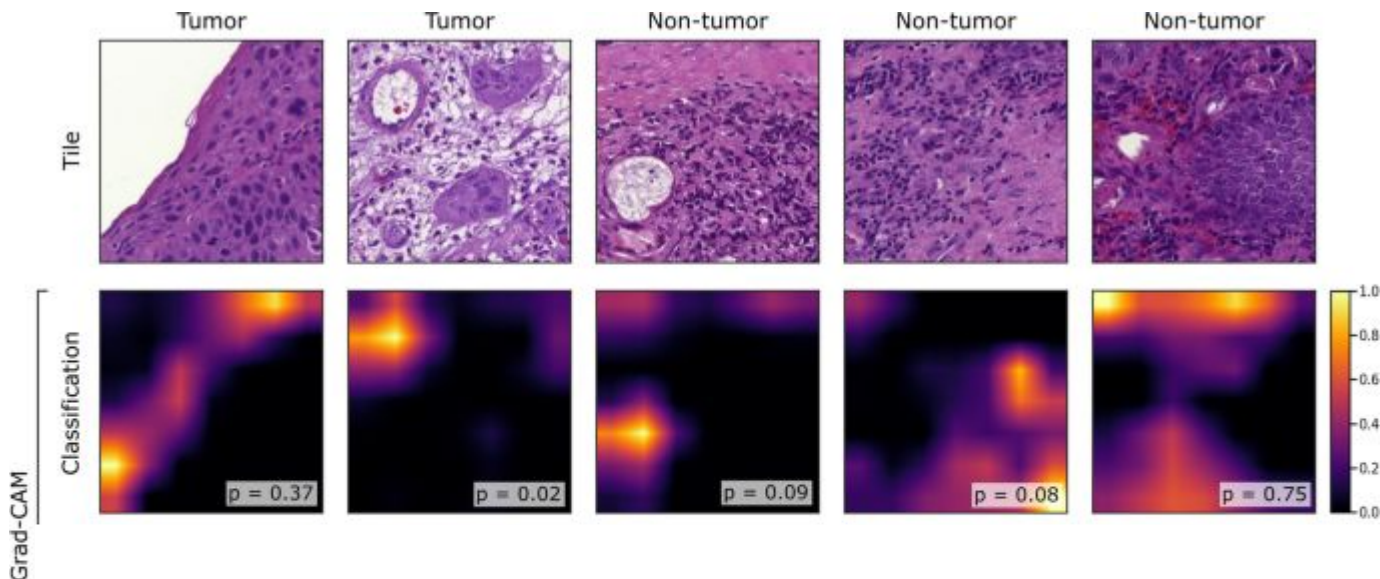


Classification / Regression / Segmentation

Examples of histology processing



Black box! Needed: Explainability / Interpretability!



Example of saliency visualisation using grad-CAM method
(see next lecture)

Siamese Convolutional Neural Network

Siamese Neural Network (SNN)

A Siamese Neural Network (SNN) is a type of neural network architecture specifically designed to **compare two inputs and determine their similarity**.

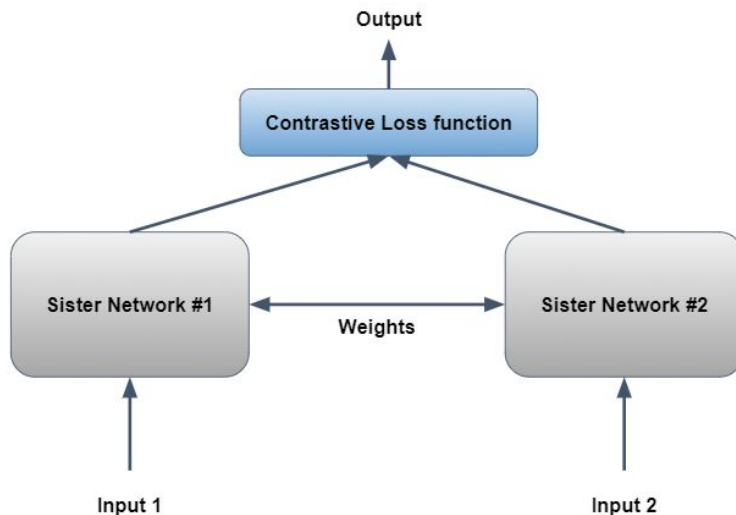
The network consists of two identical subnetworks CNN that process the inputs independently but in parallel.

The outputs of these subnetworks are then compared using a distance metric, allowing the network to learn whether the inputs are similar or dissimilar.

SNNs are particularly useful in tasks where pairwise comparison is needed, such as in face recognition, signature verification, and one-shot learning.

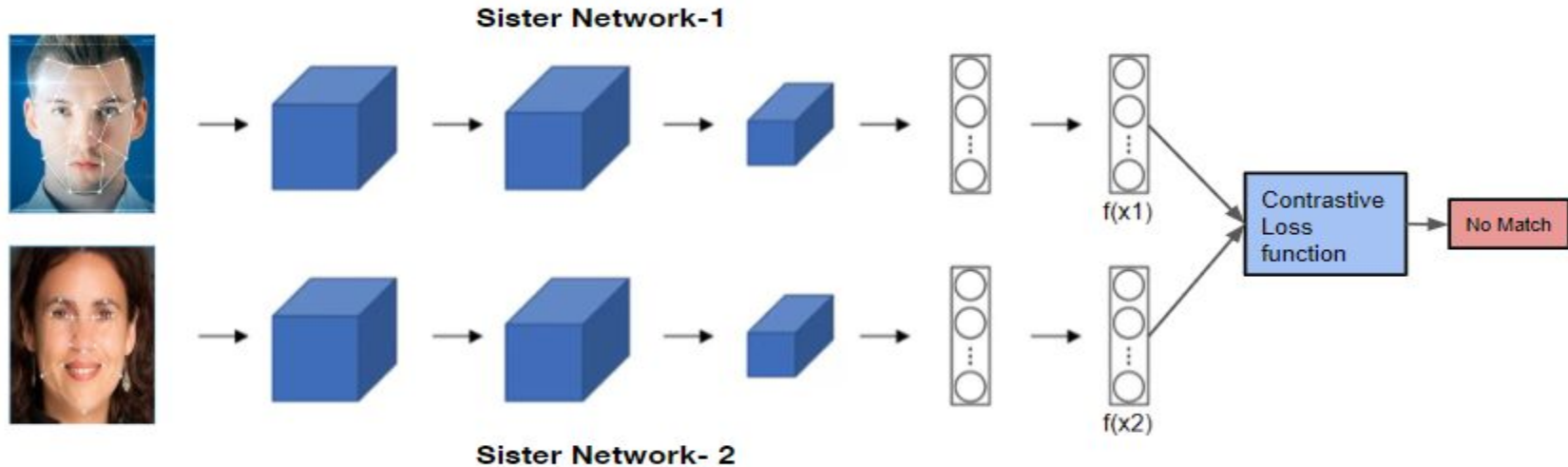
Siamese Neural Network

- Consists of two identical neural networks, also called sister networks.
- Each of the two sister networks takes one of the two input images as input.
- The outputs from the last layers are then fed into a contrastive loss function calculation, which calculates the similarity or distance between the two input images.

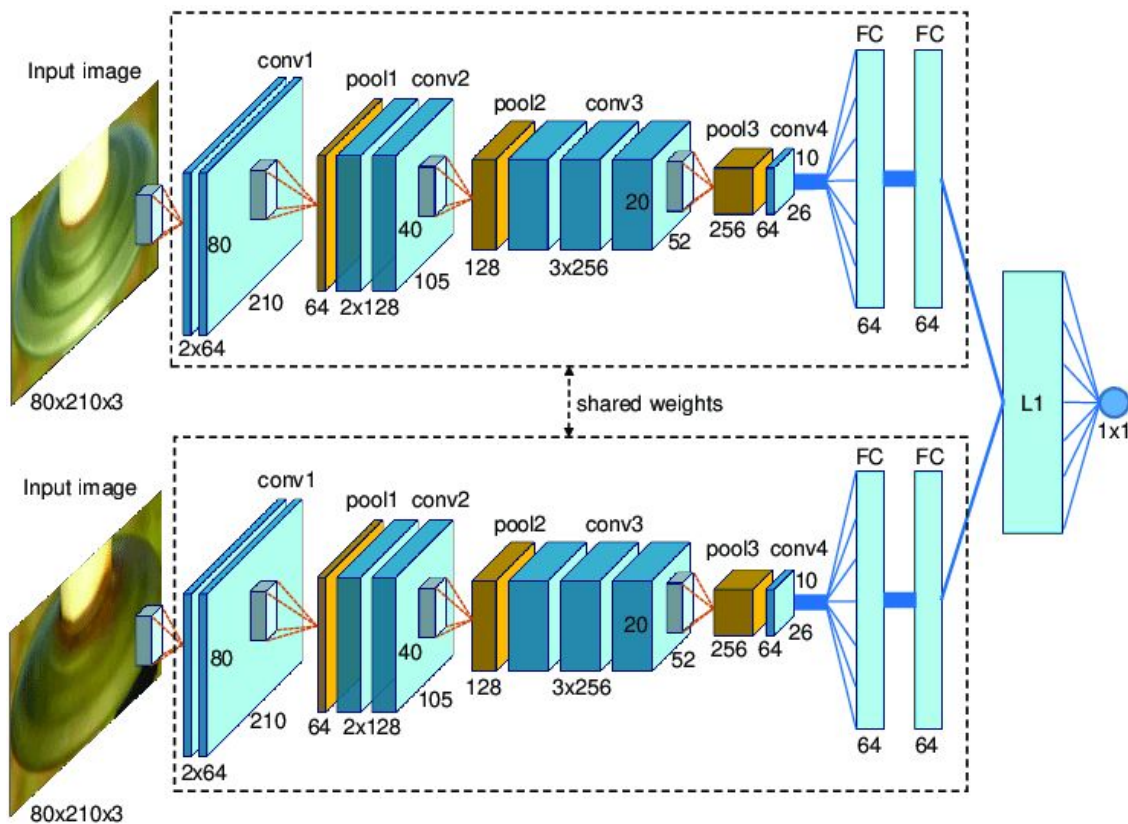


Siamese Network

Face recognition



Example of SNN Architecture



Key Features of Siamese Neural Network

- Identical Sub-networks
 - A defining characteristic of Siamese Neural Networks is the use of identical subnetworks for processing each input. These subnetworks have the same architecture and parameters, ensuring that both inputs are transformed in the same way.
- Shared Weights
 - The identical subnetworks in an SNN share the same weights.
- Learning Similarity
 - SNNs are designed to learn a similarity function that can distinguish between similar and dissimilar pairs.
 - During training, the network adjusts its weights to minimize the distance for similar pairs and maximize the distance for dissimilar pairs.

Contrastive Loss

Minimize the distance between the outputs of similar pairs and maximize the distance between the outputs of dissimilar pairs.

The contrastive loss function is defined as:

$$L = 1/2 ((1 - y) D^2 + y \max(0, m - D)^2)$$

where :

y is the label indicating whether the inputs are similar (0) or dissimilar (1),

D is the distance between the feature vectors of the two inputs

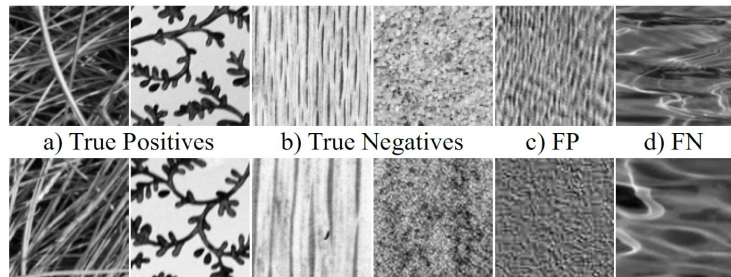
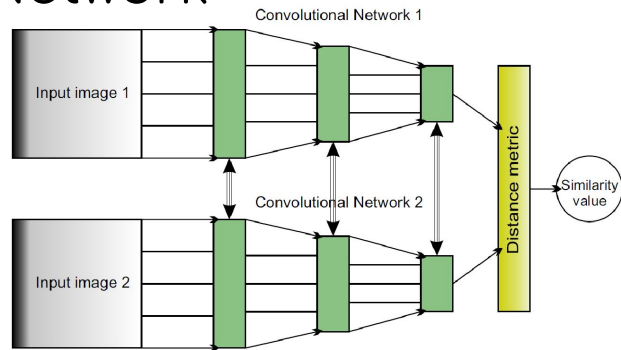
m is a margin parameter that defines the minimum distance for dissimilar pairs.

Applications of Siamese Neural Networks

- **Verification tasks:** Siamese Neural Networks (SNNs) are widely used in face verification to determine if two images belong to the same object/person. By comparing facial features, SNNs enhance security in systems like social media platforms and mobile device authentication.
- **One-shot Learning:** SNNs excel in one-shot learning, where they learn from a single example per category. This is useful in character recognition, language processing, and object classification, allowing generalization from limited data.
- **Similarity Evaluation:** SNNs measure image similarity, aiding content-based image retrieval where users search large databases using example images instead of text queries.

Texture and Convolutional Neural Network

Texture Similarity Evaluation via Siamese Convolutional Neural Network



Example Results of binary classification

General architecture of Siamese network, two networks sharing weights and metric loss

OUR NETWORK ARCHITECTURE.

Layer	Kernel Size	Stride	Output chn.
conv1	5x5	3x3	96
pool1	3x3	3x3	
conv2	5x5	1x1	265
pool2	3x3	2x2	
conv3, conv4	3x3	1x1	384
conv5	3x3	1x1	256
conv-fc1	1x1	1x1	1024
conv-fc2	1x1	1x1	128

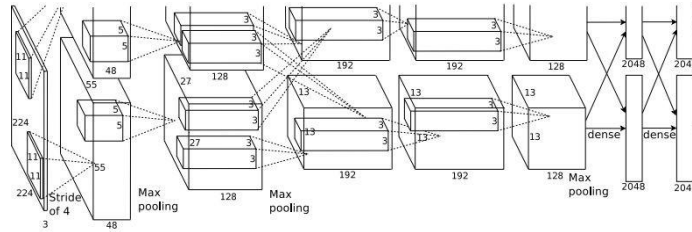
Object Detection as Classification

Multiple object -multiple instances

Object Detection as Classification

Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

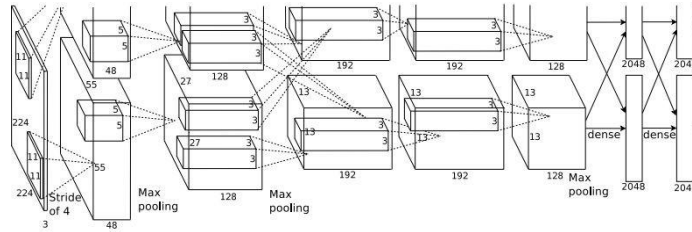


Dog? NO
Cat? NO
Background? YES

Object Detection as Classification

Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

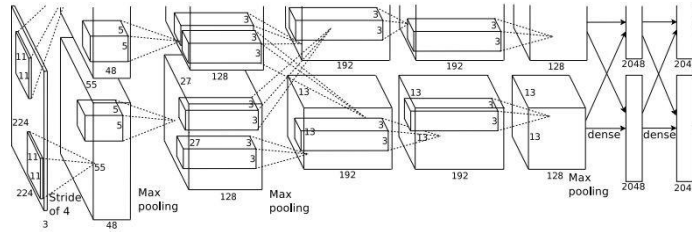


Dog? YES
Cat? NO
Background? NO

Object Detection as Classification

Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

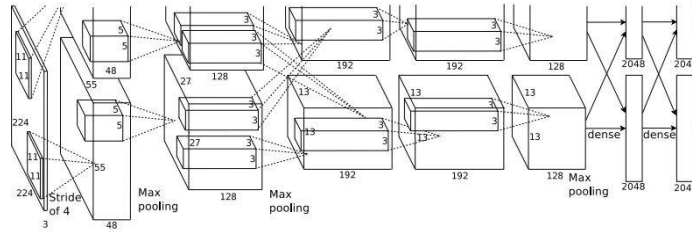


Dog? YES
Cat? NO
Background? NO

Object Detection as Classification

Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

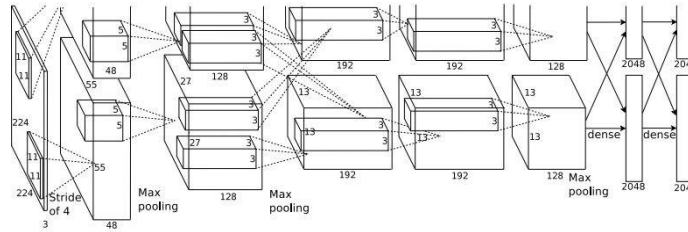


Dog? NO
Cat? YES
Background? NO

Object Detection as Classification

Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

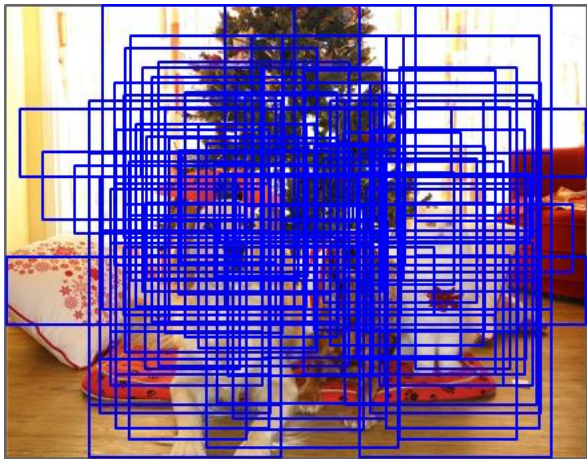


Dog? NO
Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations and scales, very computationally expensive!

Object Detection: Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

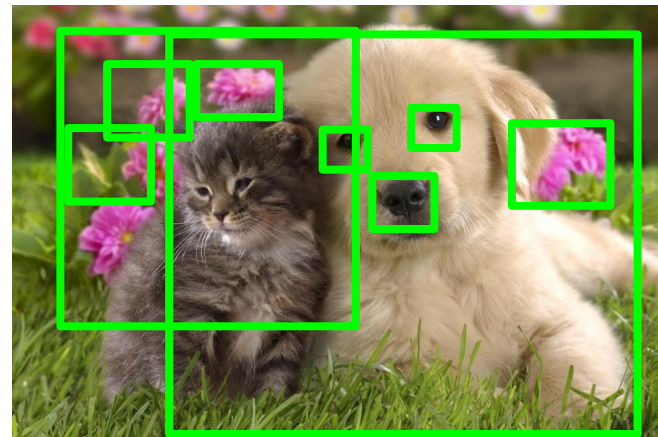


Dog? NO
Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations, scales, and aspect ratios, very computationally expensive!

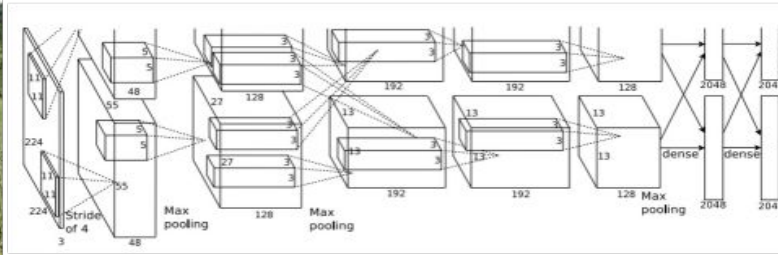
Region - crops proposals

- Find “blobby” image regions that are likely to contain objects
- “Selective Search” gives 1000 region proposals



Object Detection Classification + Localization

Single Object



Fully
Connected:
4096 to 1000

Class Scores
Cat: 0.9
Dog: 0.05
Car: 0.01
...

Vector:
4096

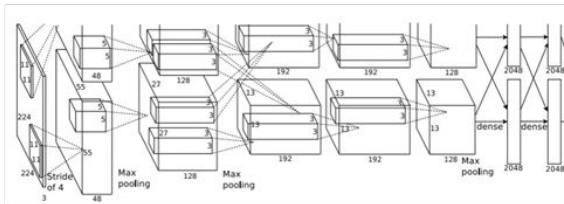
Fully
Connected:
4096 to 4

Box
Coordinates
(x, y, w, h)

Object Detection Classification + Localization

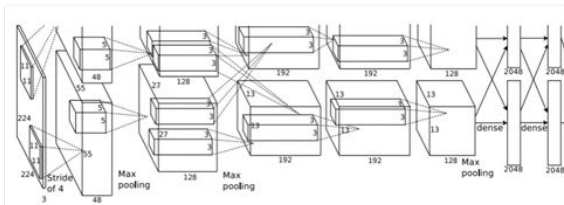
Multiple Objects

Each image needs a different
number of outputs!



CAT: (x, y, w, h)

4 numbers

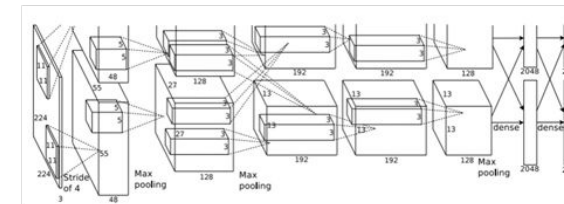


DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

12 numbers



DUCK: (x, y, w, h)

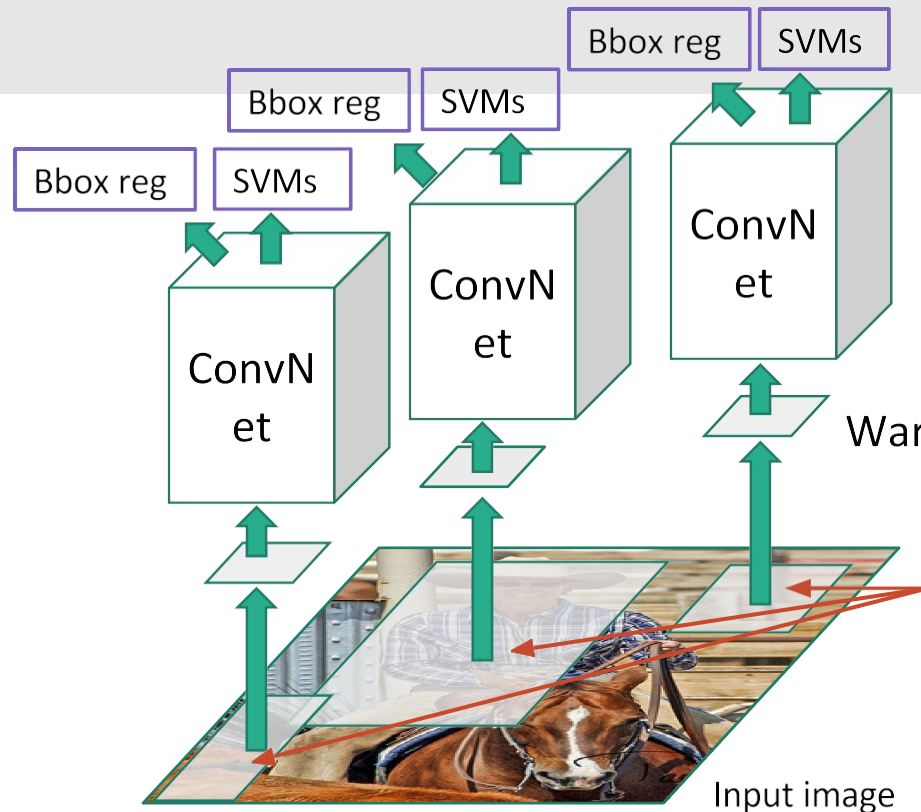
DUCK: (x, y, w, h)

Many
numbers!

...

Object Detection as Classification: R-CNN (Region CNN)

R-CNN



Linear Regression for bounding box offsets

Classify regions with SVMs

Forward each region through ConvNet

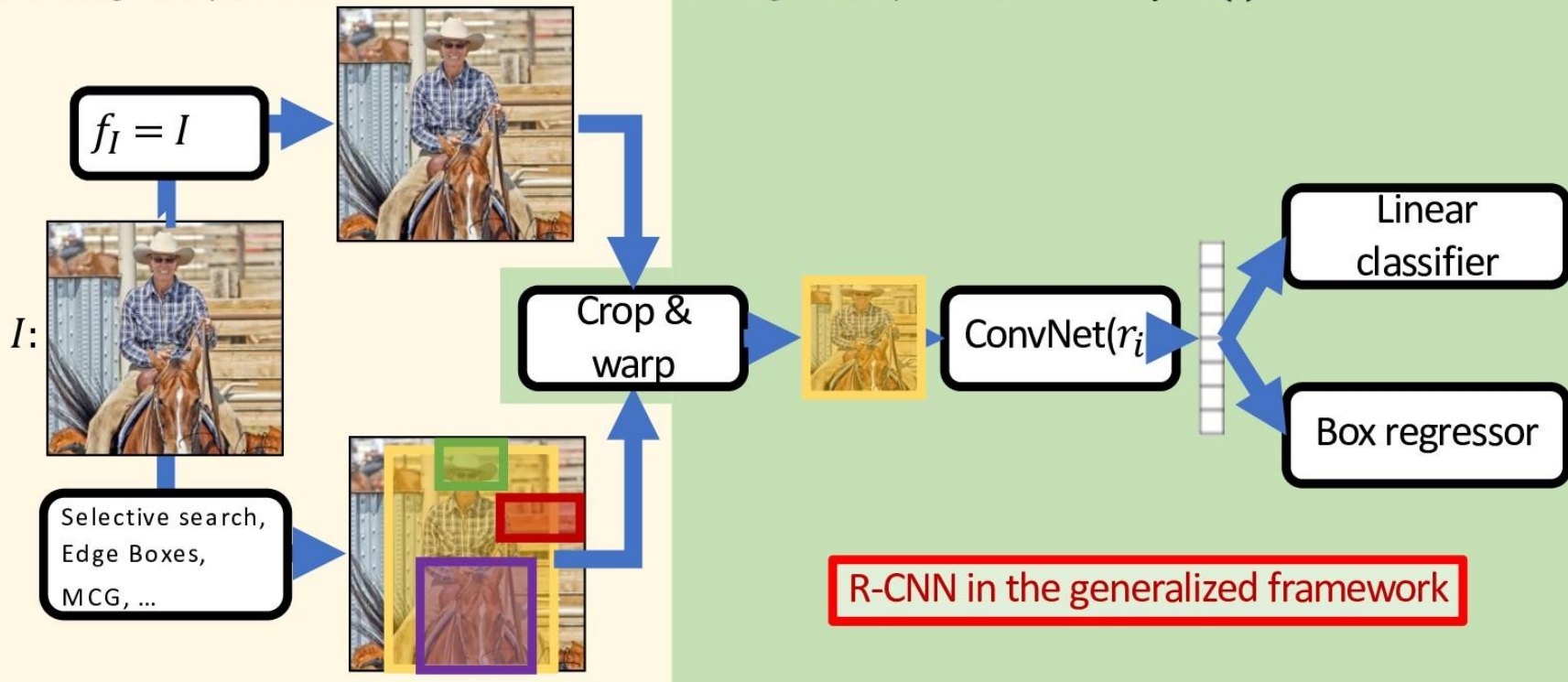
Region of Interests (ROI) from a proposal method like "Selective Search" or a "dedicated Region Proposal Network! (RPN) to propose candidate bounding boxes for objects in an image

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure :Ross Girshick, 2015;

R-CNN in Generalized Framework

Per-image computation

Per-region computation for each $r_i \in r(I)$



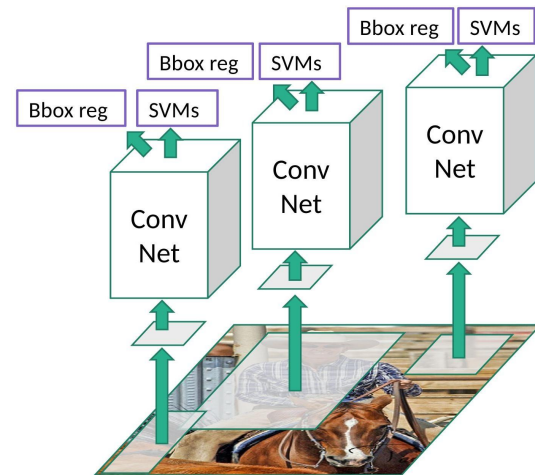
R-CNN: Objectives and Problems

Ad hoc training objectives

- Fine-tune network with softmax classifier
- Train post-hoc linear SVMs
- Train post-hoc bounding-box regressions

Problems:

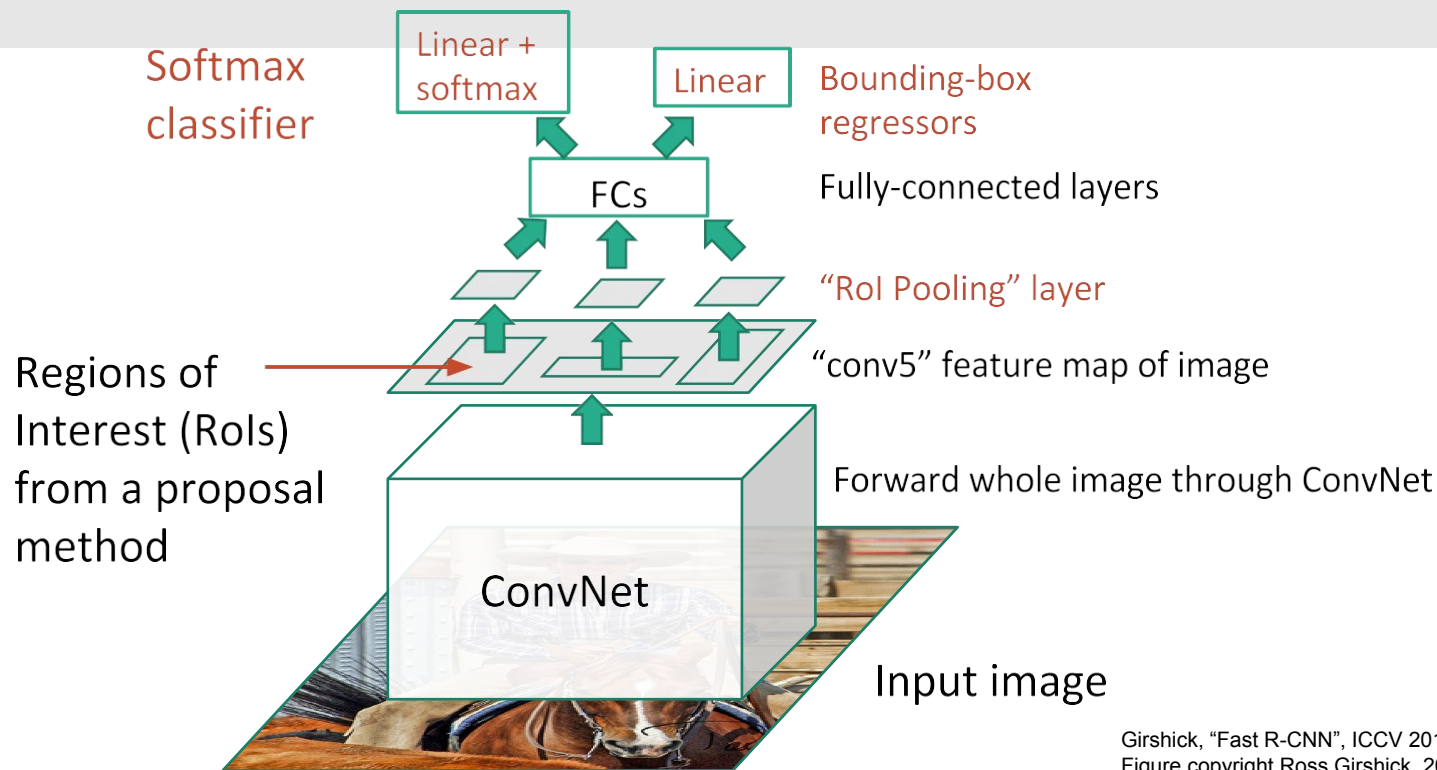
- Heavy per-region computation (e.g., 2000 full network evaluations) -> Inference (detection) is slow
- No computation/feature sharing - Training is slow, takes a lot of disk space
- Slow region proposal method adds to runtime
- Generic region proposal techniques have limited recall - the object could be overseen



Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Slide copyright Ross Girshick, 2015;

Object Detection as Classification: Fast R-CNN (Fast Region CNN)

Fast R-CNN

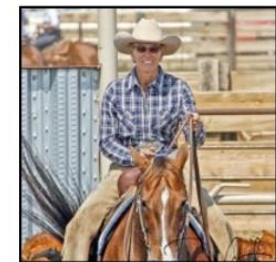


Girshick, "Fast R-CNN", ICCV 2015.
Figure copyright Ross Girshick, 2015; [source](#). Reproduced with permission.

Fast R-CNN

Region-of-Interest (RoI) Pooling on each Proposal:

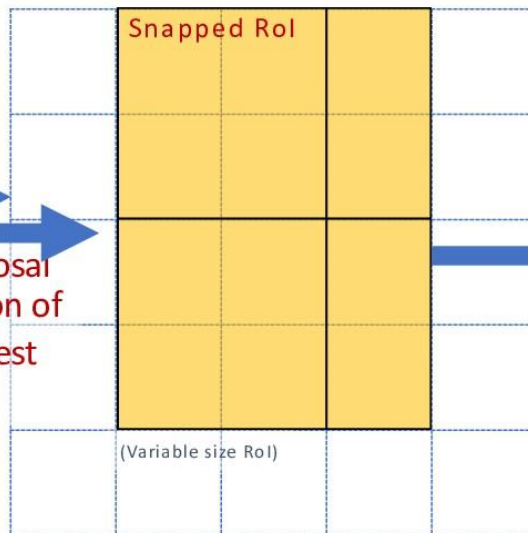
Transform **arbitrary size proposal** into a **fixed-dimensional** representation (e.g., 2x2)



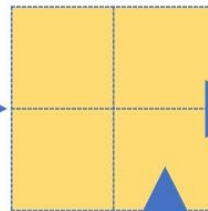
$$f_I = \text{FCN}(I)$$



Proposal
Region of
Interest
(RoI)



(Fixed dimensional
representation)

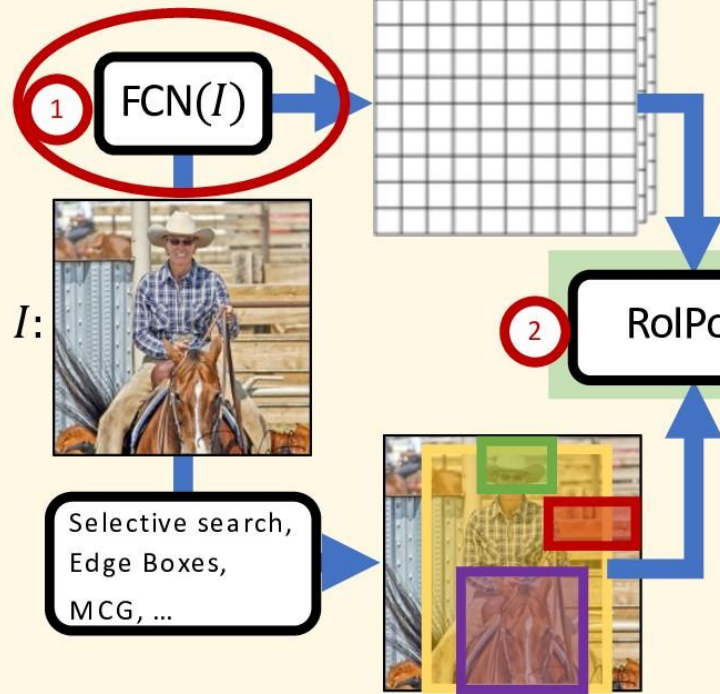


Feature value
is **max** over input
cells

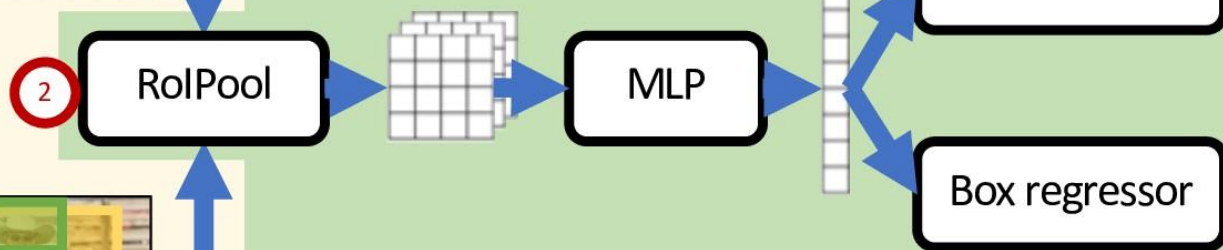
MLP

Fast R-CNN

Per-image computation



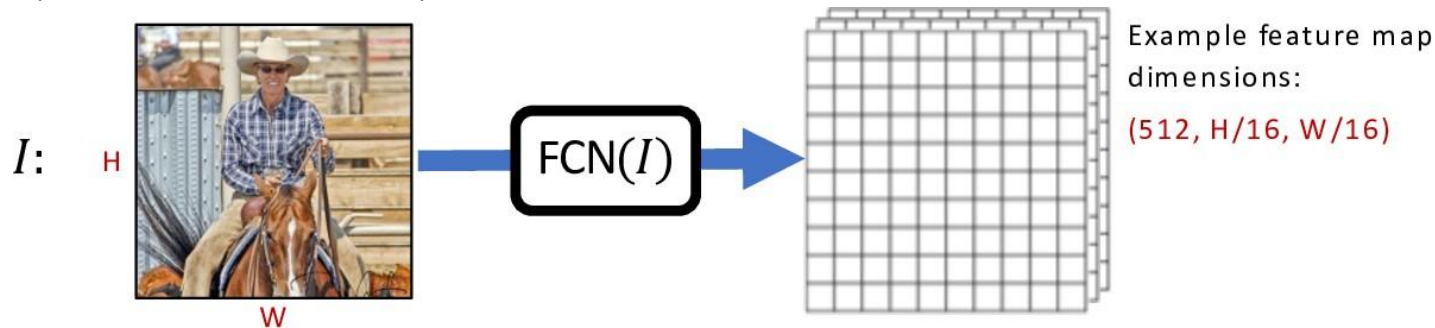
Per-region computation for each $r_i \in r(I)$



Fast R-CNN

Backbone:

- Use any standard convolutional network as “backbone” architecture
 - E.g.: AlexNet, VGG, ResNet, Inception, ResNeXt, DenseNet, ...
- Remove global pooling $\Rightarrow$ output spatial dims proportional to input spatial dims
- A good network exploits the strongest recognition backbone (features matter!)



Fast R-CNN

What is the problem with Fast R-CNN?

- ~~Heavy per region computation (e.g., 2000 full network evaluations)~~
- ~~No computation/feature sharing~~
- Slow region proposal method adds to runtime
- Generic region proposal techniques have low recall

Object Detection as Classification: Faster R-CNN (Faster Region CNN)

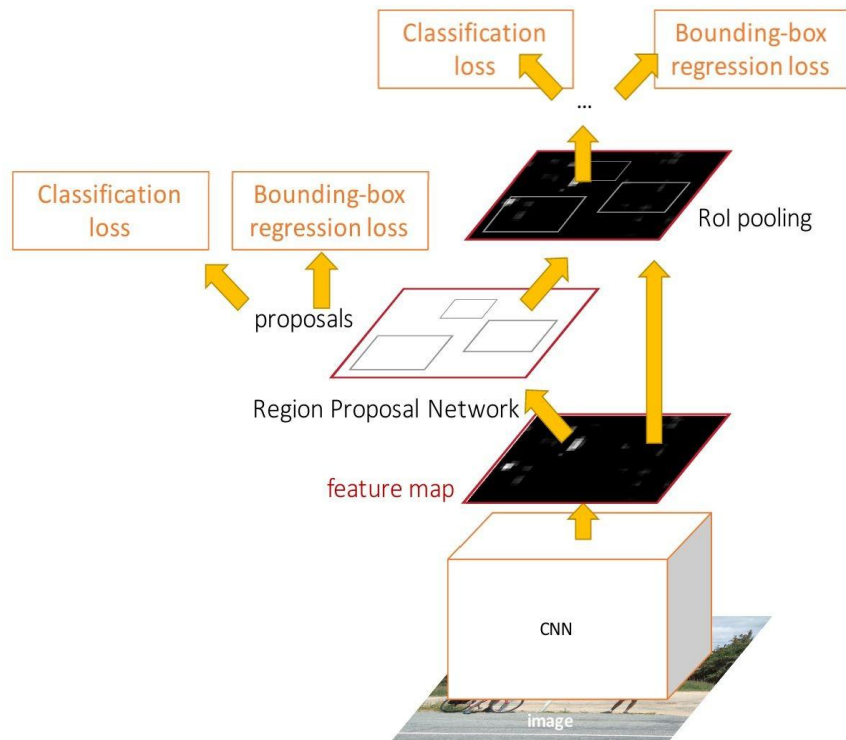
Faster R-CNN

Make CNN do proposals!

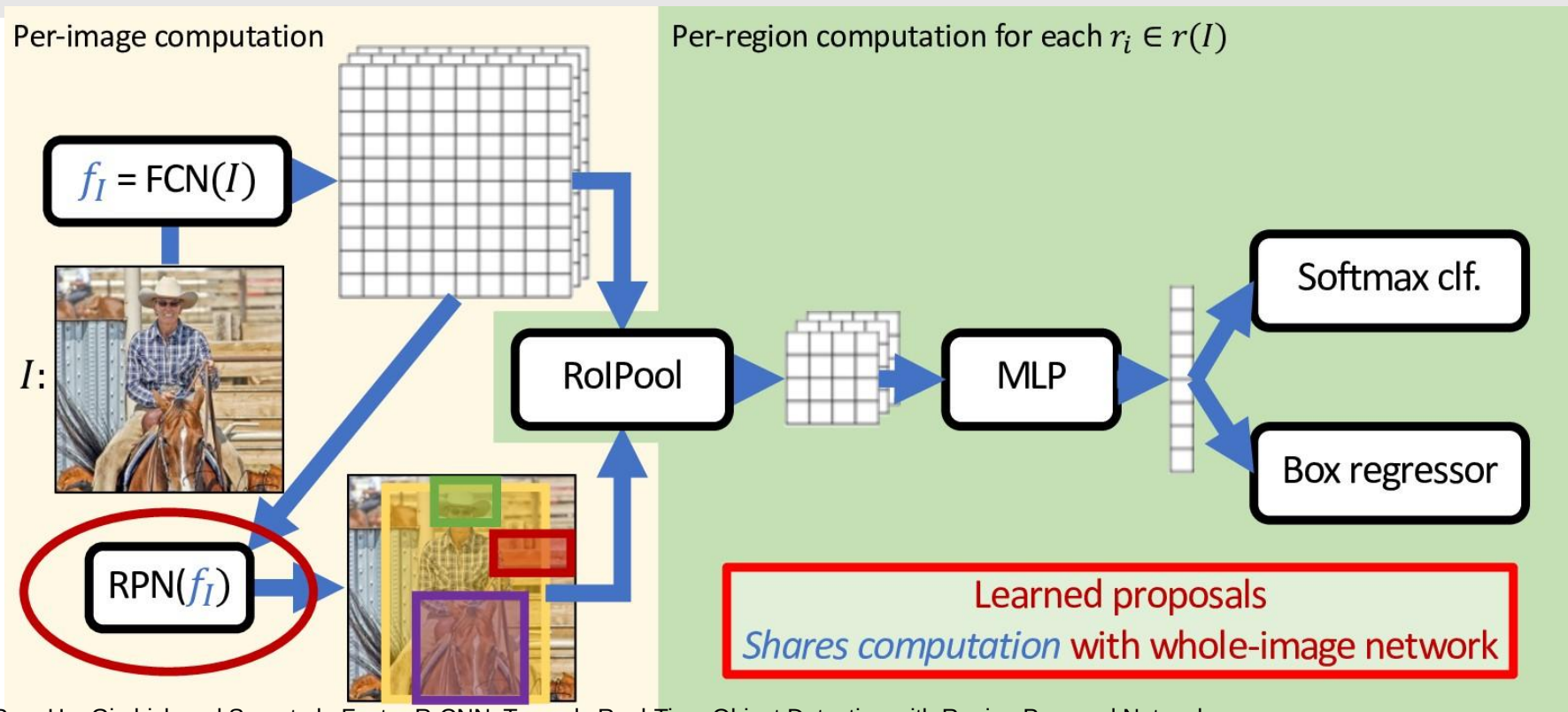
Insert Region Proposal Network (RPN) to predict proposals from features

Jointly train with 4 losses:

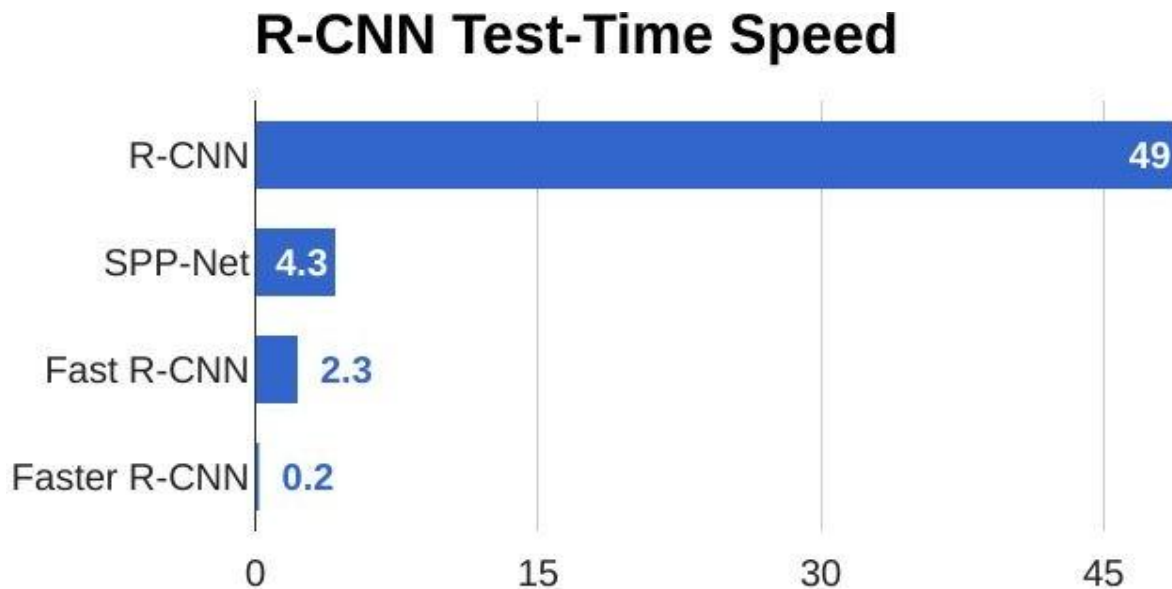
1. RPN classify object / not object
2. RPN regress box coordinates
3. Final classification score (object classes)
4. Final box coordinates



Faster R-CNN



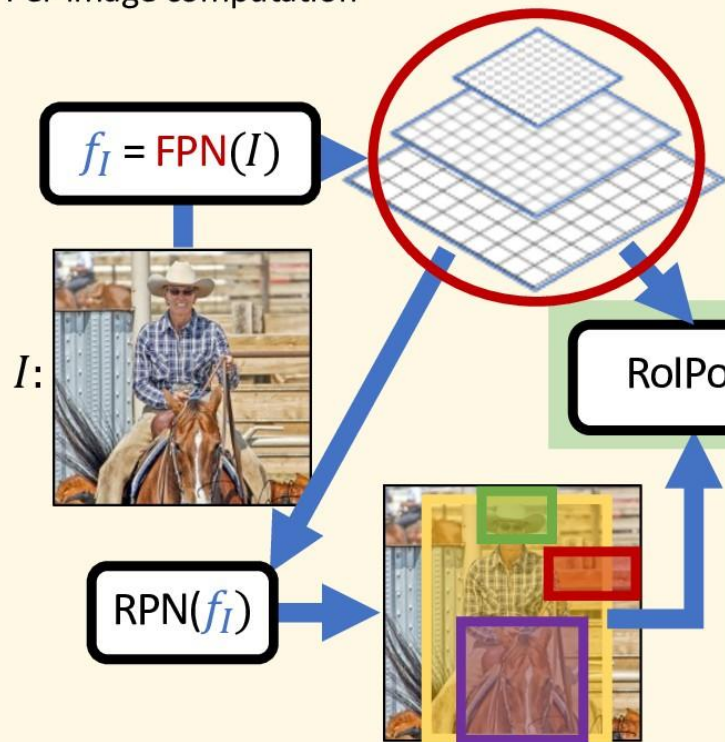
R-CNN Test Time Speed



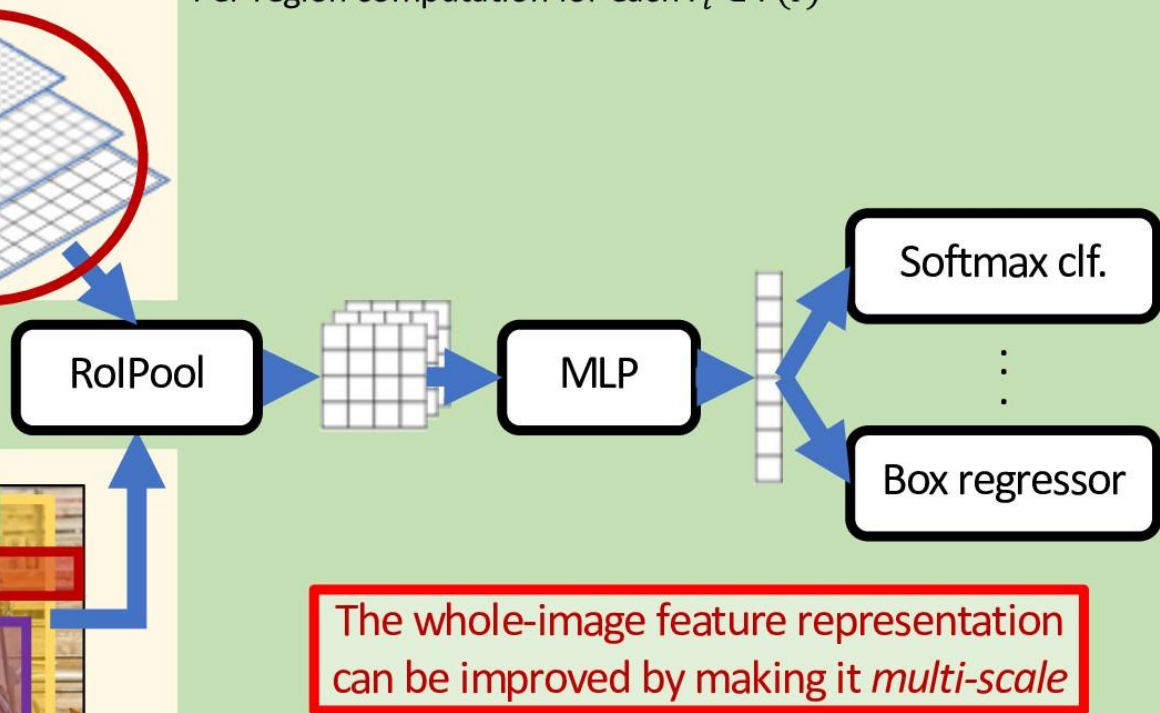
Feature Pyramid Network

Feature Pyramid Network

Per-image computation

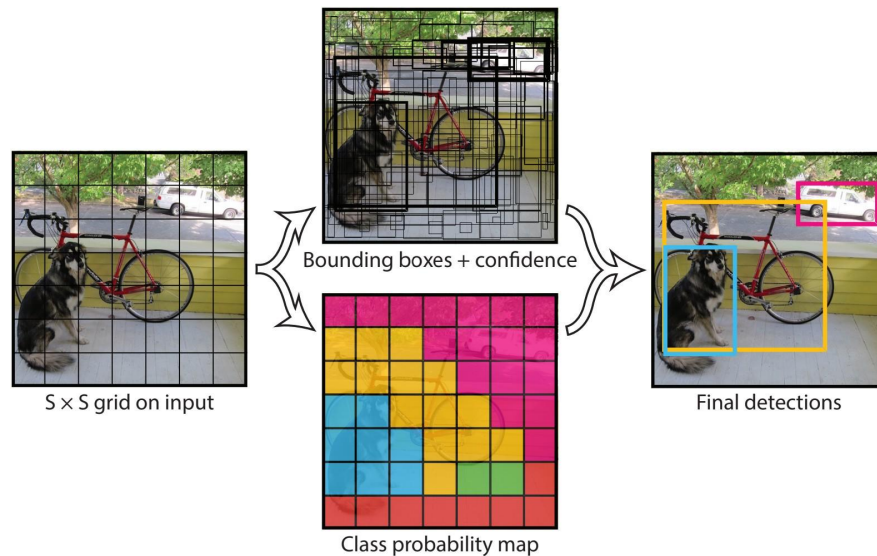


Per-region computation for each $r_i \in r(I)$



SSD (Single Shot Detector) /YOLO

Single Stage Detection



- YOLO and SSD: Alternative to proposal based (two stage) methods
- Predict 2D bounding boxes in a single stage
- Faster, but often worse performance in practice

YOLO "You Only Look Once"

YOLO — You Only Look Once

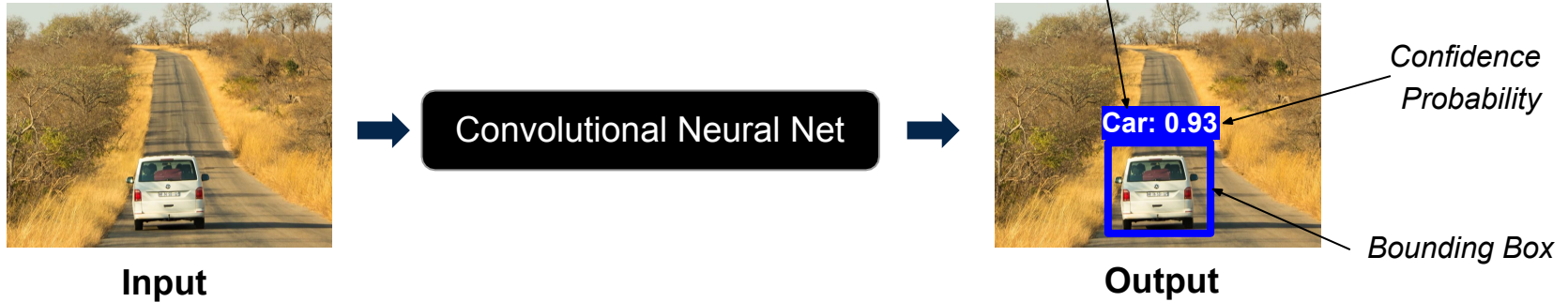
- Take an image and split it into an $S \times S$ grid, within each of the grid we take m bounding boxes.
- For each of the bounding box, the network outputs a class probability and offset values for the bounding box.
- The bounding boxes having the class probability above a threshold value is selected and used to locate the object within the image.

In YOLO a single convolutional network predicts the bounding boxes and the class probabilities for these boxes.

YOLO "You Only Look Once"

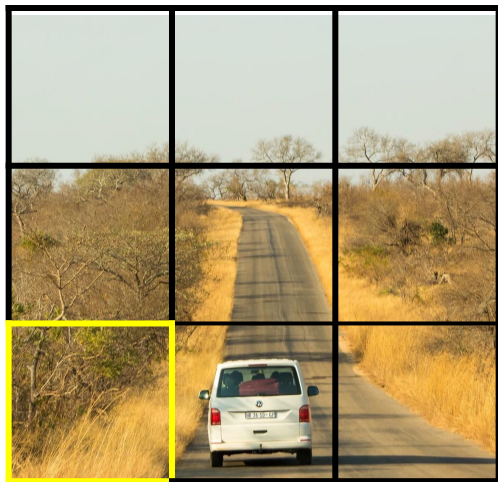
Instead of making predictions on many regions of an image, YOLO passes the entire image at once into a CNN that predicts the labels, bounding boxes, and confidence probabilities for objects in the image.

YOLO runs much faster than region based algorithms quick because requires only a single pass through a CNN.



YOLO Steps

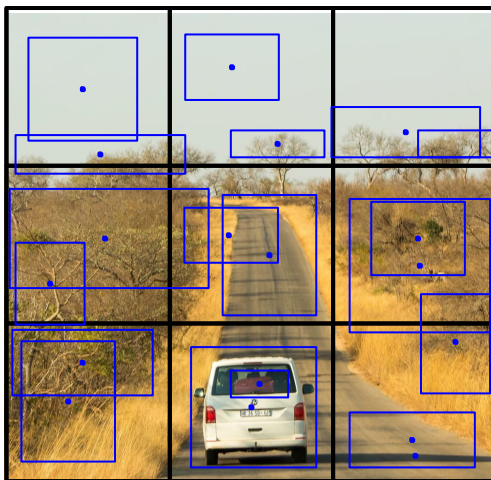
1. Divide the image into cells with an $S \times S$ grid.



Cell

$S = 3$

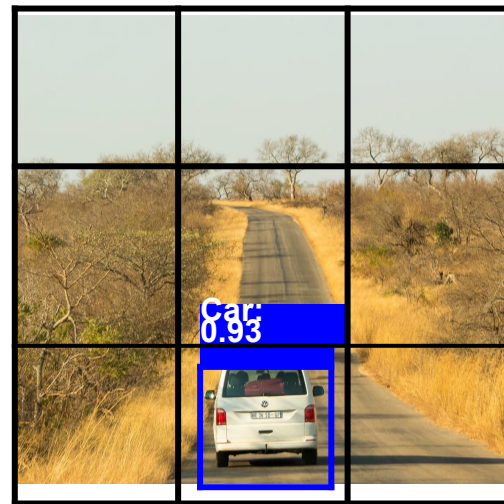
2. Each cell predicts B bounding boxes.



$B = 2$

A cell is responsible for detecting an object if the object's bounding box falls within the cell. (Notice that each cell has 2 blue dots.)

3. Return bounding boxes above confidence threshold.

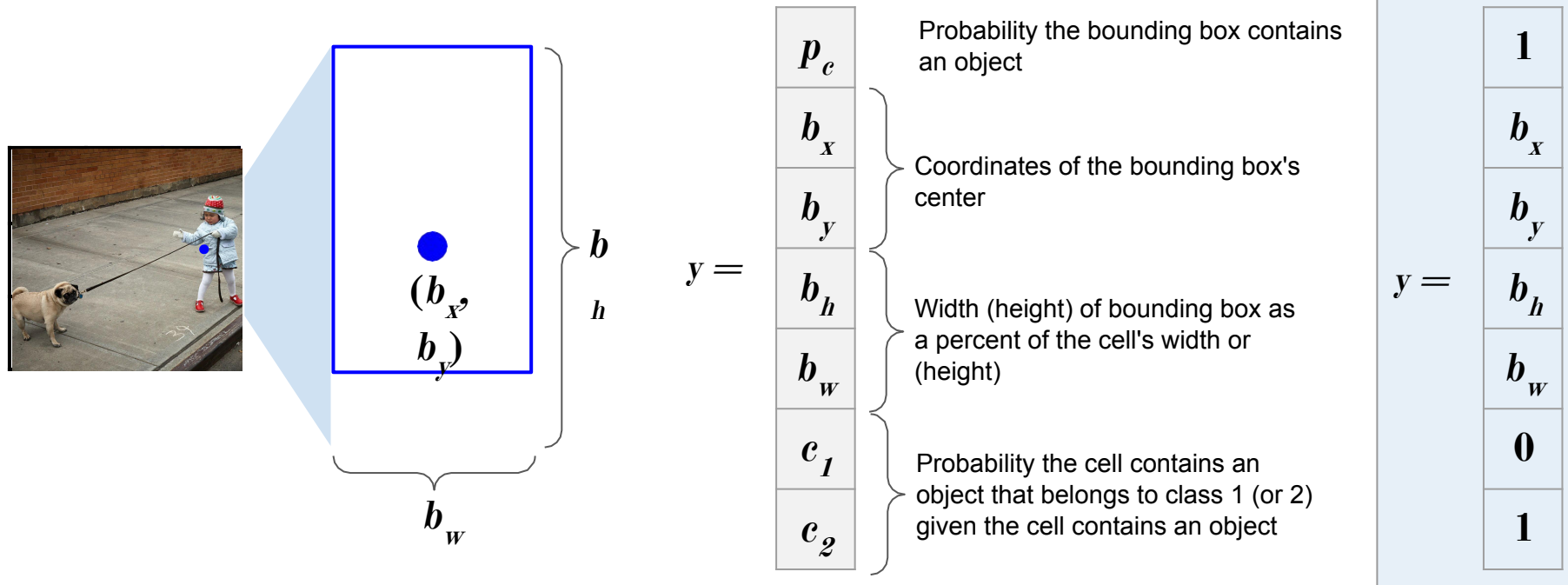


All other bounding boxes have a confidence probability less than the threshold (say 0.90) so they are suppressed.

In practice, we would use large values ($S = 19$ and $B = 5$) to identify more objects.

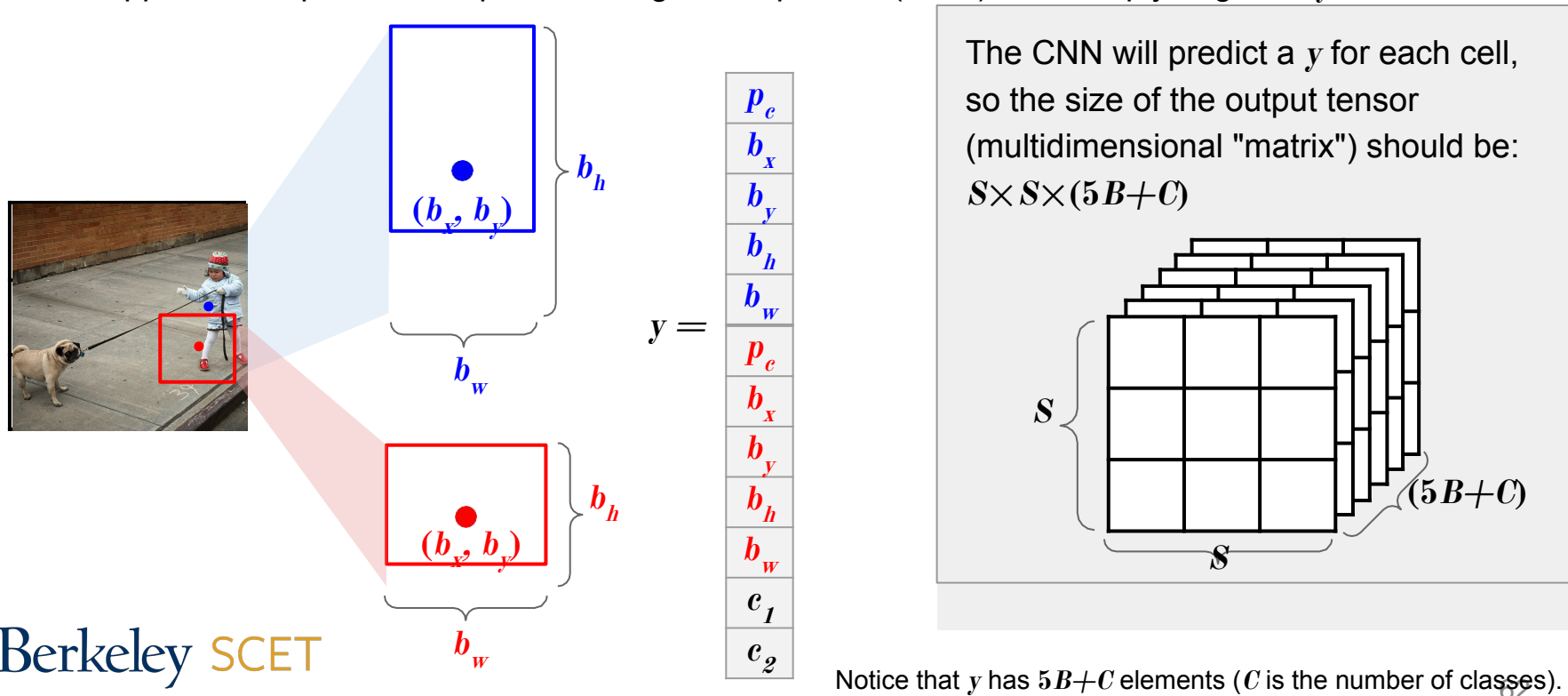
How are bounding boxes encoded?

Let's use a simple example where there are 3×3 cells ($S=3$), each cell predicts 1 bounding box ($B=1$), and objects are either dog = 1 or human = 2. For each cell, the CNN predicts a vector y :

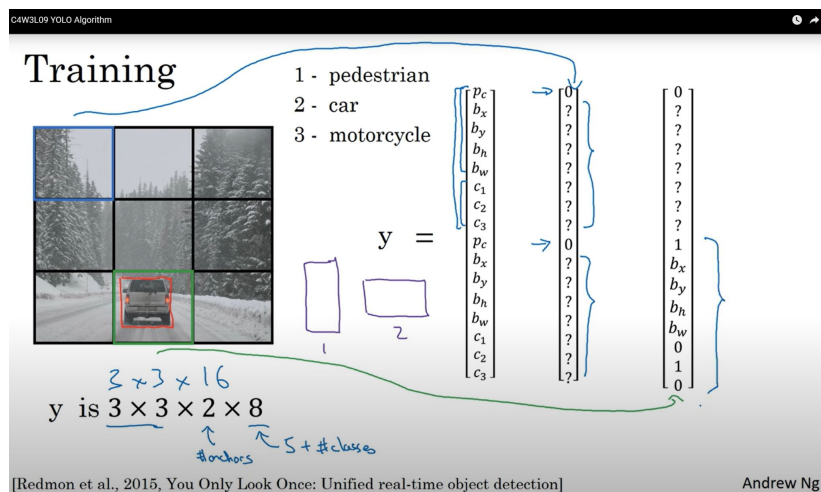


Encoding Multiple Bounding Boxes

What happens if we predict multiple bounding boxes per cell ($B > 1$)? We simply augment y .



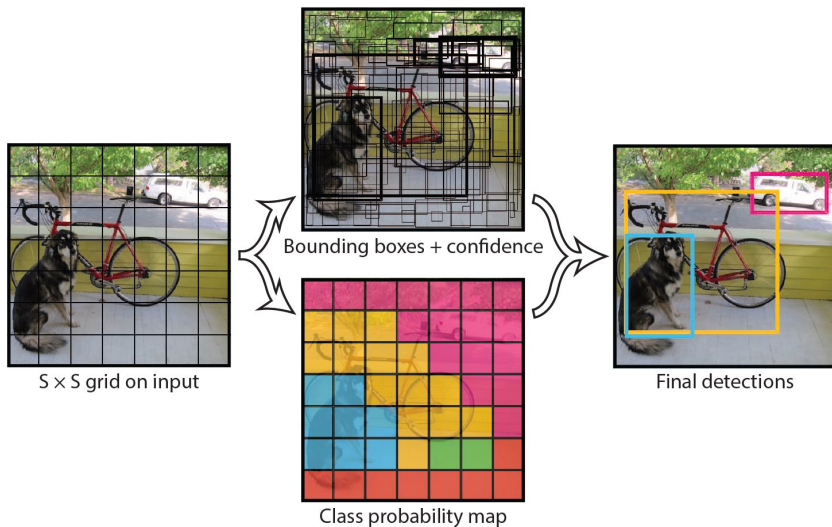
Yolo - Andrew Ng presentation



Andrew Ng

https://www.youtube.com/watch?v=9s_FpMpdYW8&a_b_channel=DeepLearningAI

Yolo - original paper



The system divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object. Each grid cell predicts B bounding boxes and confidence scores for those boxes. If no object exists in that cell, the confidence scores should be zero. Otherwise we want the confidence score to equal the intersection over union (IOU) between the predicted box and the ground truth.

Each bounding box consists of 5 predictions: x , y , w , h , and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell. The width and height are predicted relative to the whole image. Finally the confidence prediction represents the IOU between the predicted box and any ground truth box.

Each grid cell also predicts C conditional class probabilities, $\Pr(\text{Class}_i | \text{Object})$. These probabilities are conditioned on the grid cell containing an object.

We only predict one set of class probabilities per grid cell, regardless of the number of boxes B . At test time we multiply the conditional class probabilities and the individual box confidence predictions,

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

which gives us class-specific confidence scores for each box. These scores encode both the probability of that class appearing in the box and how well the predicted box fits the object.

* IOU (Intersection Over Union)

YOLO Overview

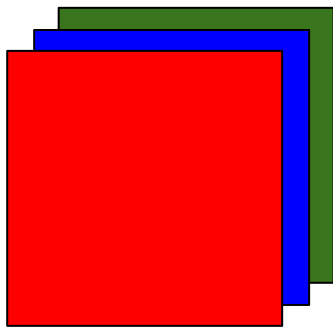
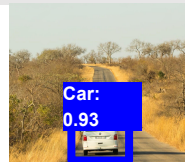
Input



Convolutional Neural Net

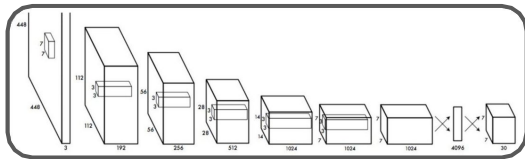


Output

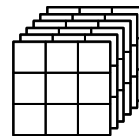


$$W \times H \times 3$$

W : Width of image in pixels
 H : Height of image in pixels
3: Number of color channels in RGB



Series of convolutional and pooling layers.

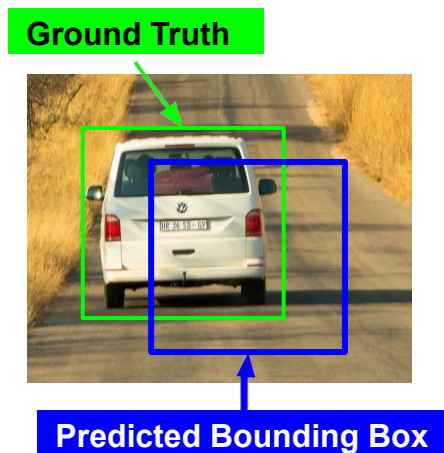


$$S \times S \times (5B + C)$$

A tensor that specifies the bounding box locations and class probabilities.

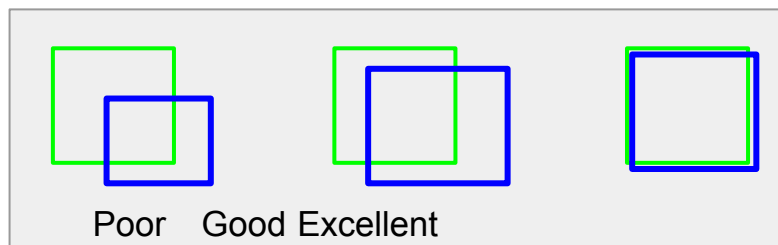
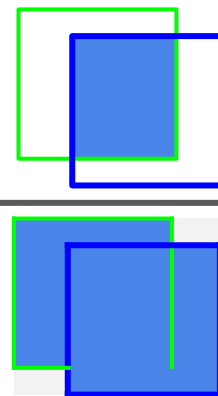
Measuring Performance with Uol

- Union over Intersection (Uol) measures the overlap between two bounding boxes.
- During training, we calculate the Uol between a predicted bounding box and the ground truth (the pre-labeled bounding box we aim to match)



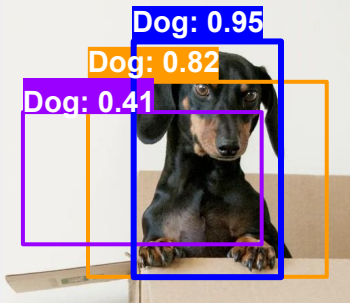
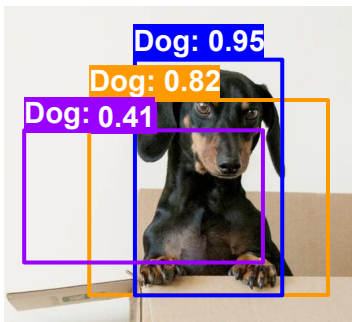
Intersection over Union

$$= \frac{\text{Area of Intersection}}{\text{Area of Union}}$$



Double Counting Objects (Non-Max Suppression)

- When predicting more than 2 bounding boxes per cell, sometimes the same object will be detected multiple times (overlapping boxes with the same label)
- **Non-max suppression** solves multiple counting by removing the box with the lower confidence probability when the UoI between 2 boxes with the same label is above some threshold.



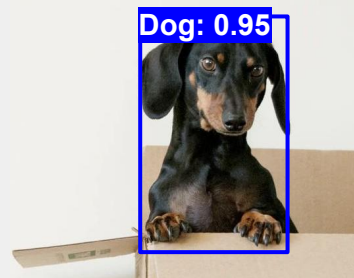
1. Identify the box with the highest confidence.

Non-Max Suppression

 UoI: 0.62

 UoI: 0.47

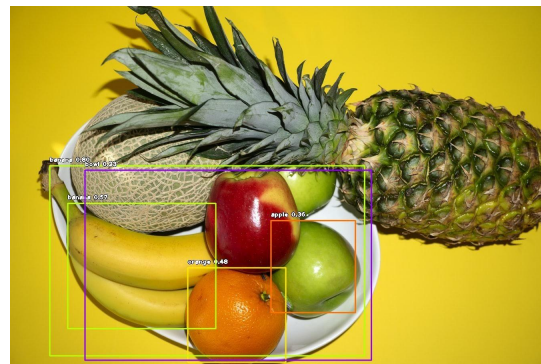
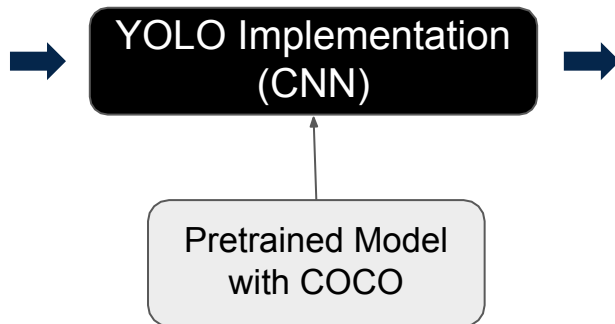
2. Calculate the UoI between the highest confidence box each of the other boxes.



3. Suppress boxes with UoI above a selected threshold (usually 0.3)

Pretrained Models

- Training a YOLO model requires images labeled with bounding boxes.
- These datasets may take time to label, so readily available prelabeled images are often used to train models.
- A common dataset for image classification/detection/segmentation is the [COCO](#) ([Common Objects in Context](#)), a database of images with 80 labelled classes.
- Popular pretrained YOLO models with COCO:
 - [ImageAI](#) (easy-to-use, lightweight YOLO implementation)
 - [Darknet](#) (trained by the author of YOLO)



Pineapples and cantaloupes are not in COCO so they are not recognized.

COCO Pretrained Labels

Applications built with COCO trained models will only be able to identify these objects!

person	fire hydrant	elephant	skis	wine glass	broccoli	dining table	toaster
bicycle	stop sign	bear	snowboard	cup	carrot	toilet	sink
car	parking meter	zebra	sports ball	fork	hot dog	tv monitor	refrigerator
motorbike	bench	giraffe	kite	knife	pizza	laptop	book
aeroplane	bird	backpack	baseball bat	spoon	donut	mouse	clock
bus	cat	umbrella	baseball glove	bowl	cake	remote	vase
train	dog	handbag	skateboard	banana	chair	keyboard	scissors
truck	horse	tie	surfboard	apple	sofa	cell phone	teddy bear
boat	sheep	suitcase	tennis racket	sandwich	potted plant	microwave	hair-drier
traffic light	cow	frisbee	bottle	orange	bed	oven	toothbrush

YOLO - demo

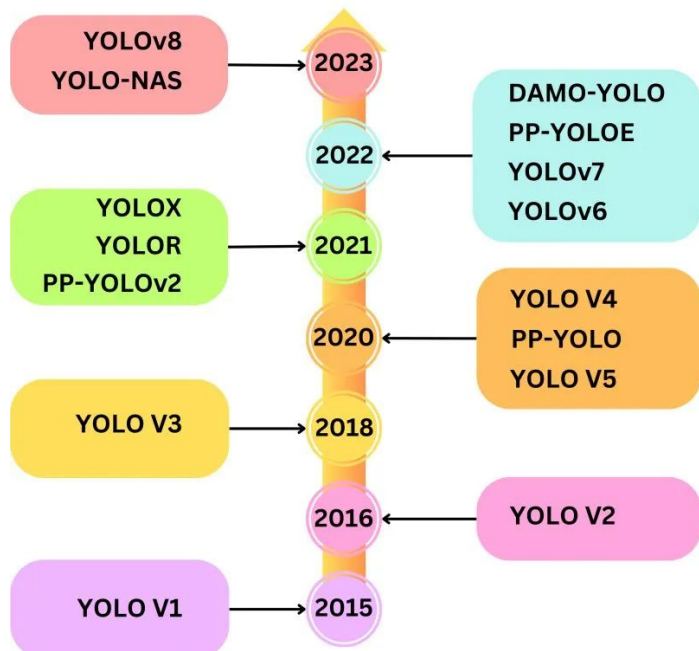


YOLO versions timeline

The YOLO framework has undergone several iterations since its initial release in 2015 by Joseph Redmon and a team of collaborators (<https://arxiv.org/pdf/1506.02640.pdf>).

The YOLO timeline

TheAiEdge.io

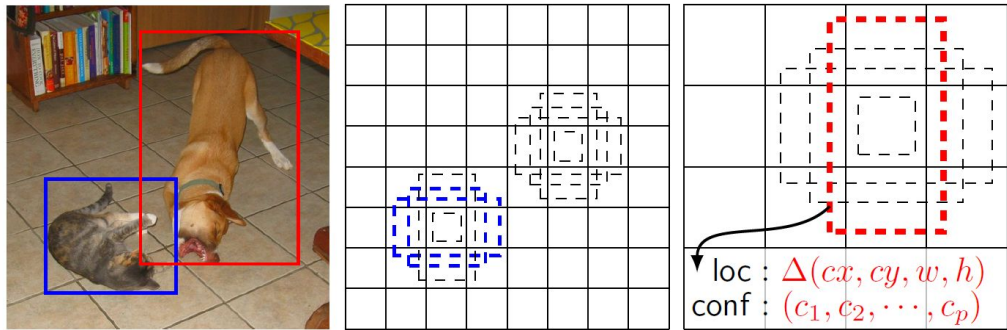


Actual:

2025: YOLOv13

SSD (Single Shot Detector)

SSD framework.

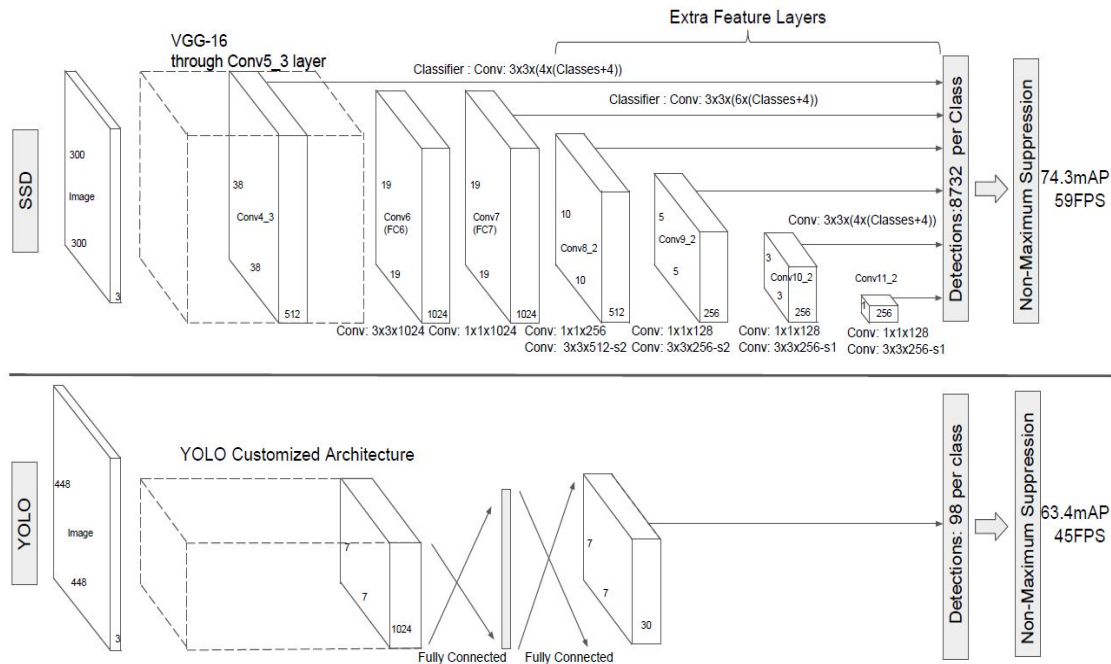


(a) Image with GT boxes (b) 8×8 feature map (c) 4×4 feature map

(a) SSD only needs an input image and ground truth boxes for each object during training. In a convolutional fashion, we evaluate a small set (e.g. 4) of default boxes of different aspect ratios at each location in several feature maps with different scales (e.g. 8×8 and 4×4 in (b) and (c)). For each default box, we predict both the shape offsets and the confidences for all object categories $((c_1; c_2; \dots; c_p))$.

At training time, we first match these default boxes to the ground truth boxes. For example, we have matched two default boxes with the cat and one with the dog, which are treated as positives and the rest as negatives. The model loss is a weighted sum between localization loss and confidence loss (e.g. Softmax).

A comparison between two single shot detection models: SSD and YOLO



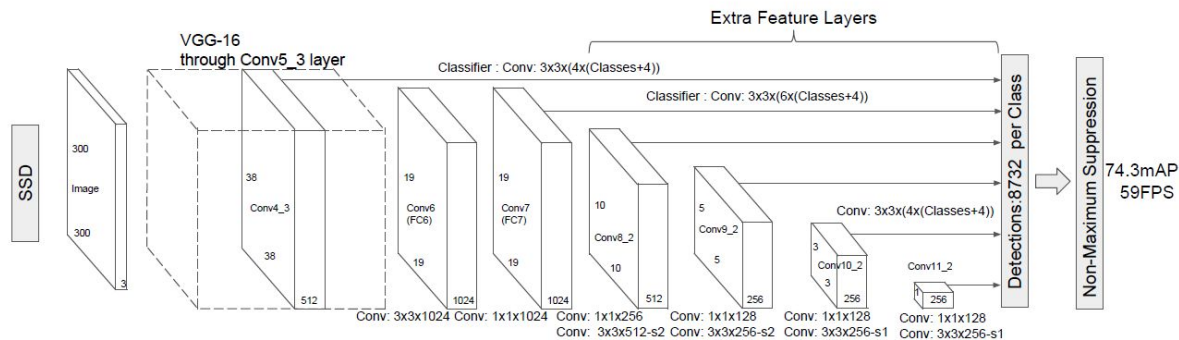
SSD with a 300x300 input size significantly outperforms its 448x448 YOLO counterpart in accuracy on VOC2007 test while also improving the speed.

SSD - Multi-scale feature maps for detection

The authors add convolutional feature layers to the end of the truncated base network.

These layers decrease in size progressively and allow predictions of detections at multiple scales.

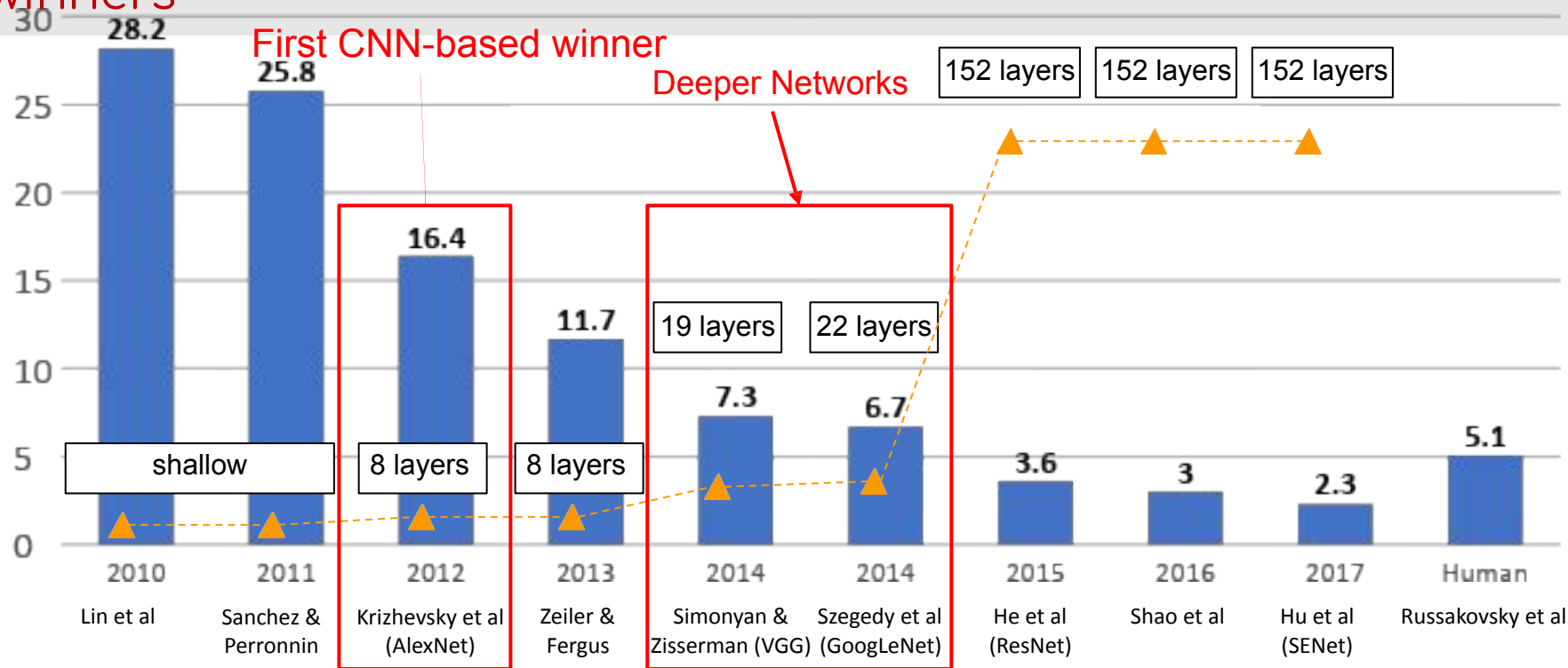
The convolutional model for predicting detections is different for each feature layer.



....YOLO operate on a single scale feature map

DNN Architectures

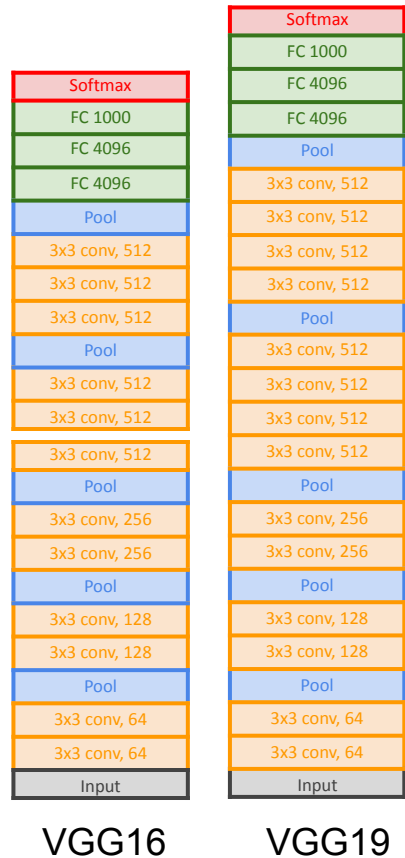
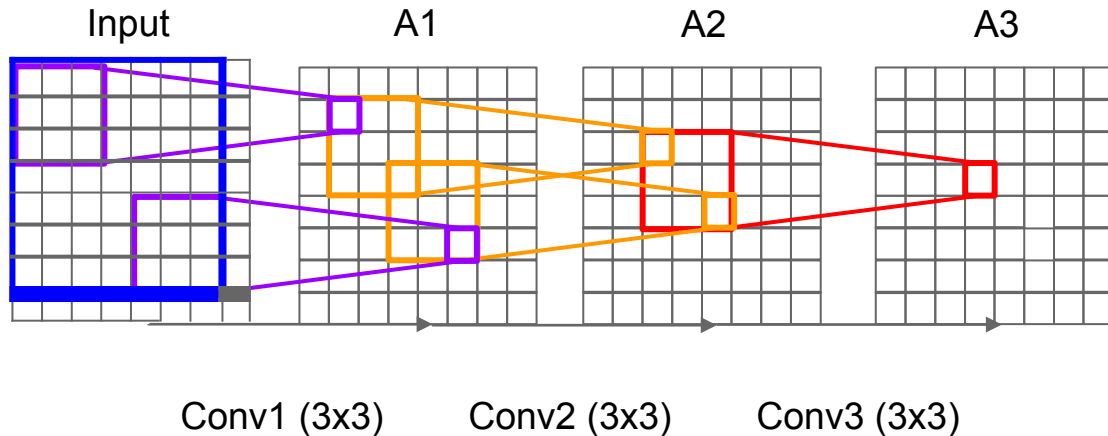
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



VGGNet

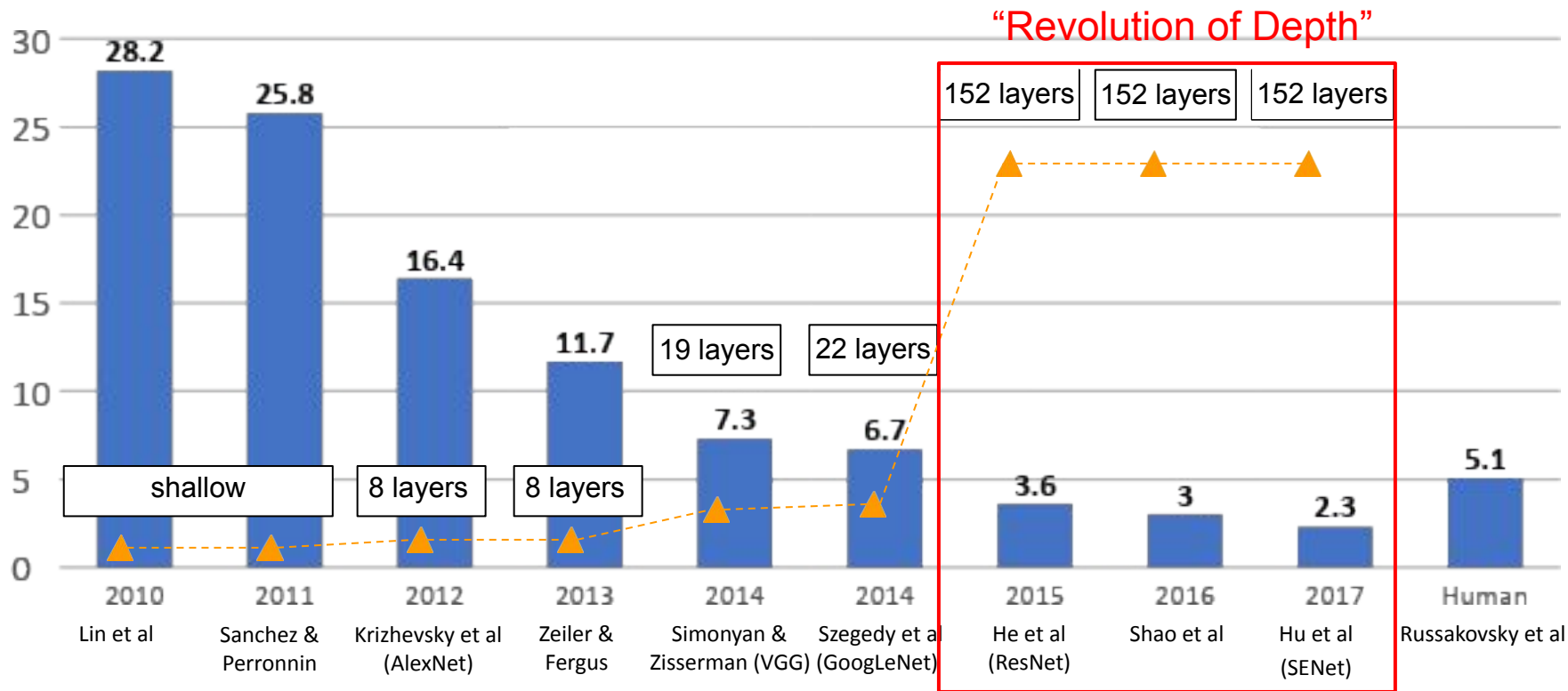
[Simonyan and Zisserman, 2014]

Q: What is the effective receptive field of three 3x3 conv (stride 1) layers?



Stack of three 3x3 conv (stride 1) layers has same **effective receptive field** as one 7x7 conv layer

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



Problem of very Deep Models

[He et al., 2015]

Fact:

Deep models have more representation power (more parameters) than shallower models.

Hypothesis:

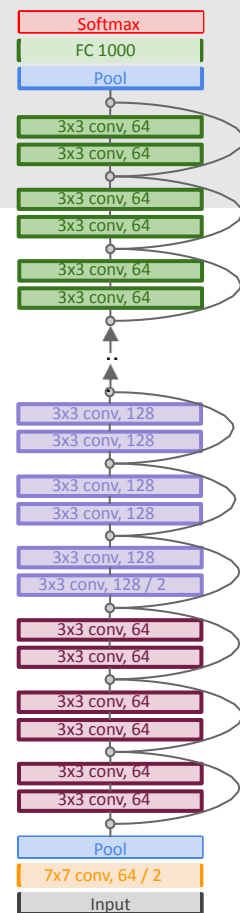
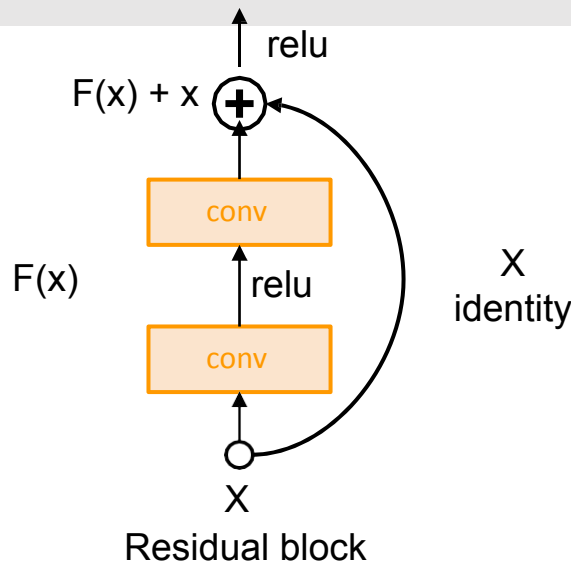
the problem is an *optimization* problem, deeper models are harder to optimize!

ResNet - residual connections

[He et al., 2015]

Very deep networks using residual connections

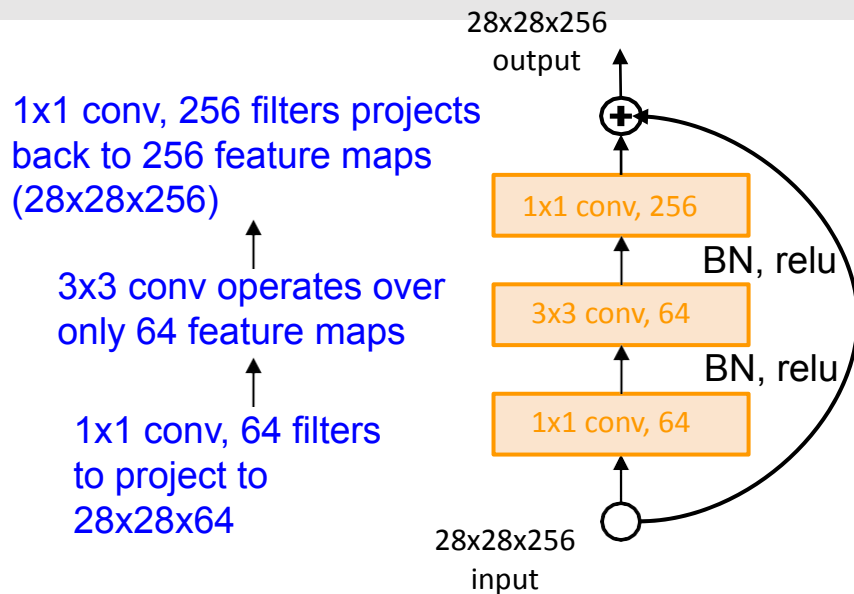
- Stack residual blocks
- Every residual block has two 3x3 conv layers
- Periodically, double number of filters and downsample spatially using stride 2 (/2 in each dimension)



ResNet

[He et al., 2015]

For deeper networks (ResNet-50+), use “bottleneck” layer to improve efficiency (similar to GoogLeNet)



ResNet

[He et al., 2015]

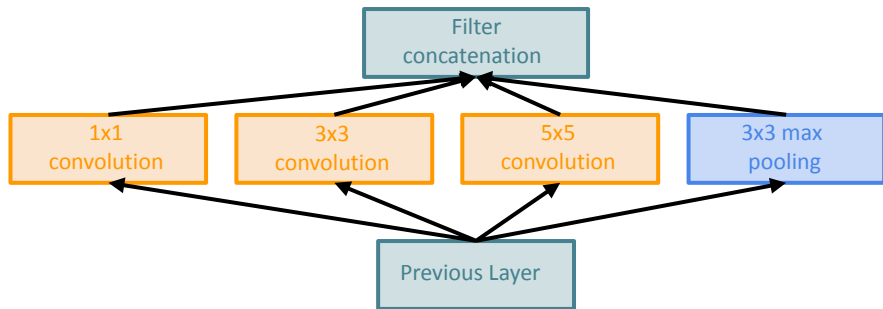
Training ResNet in practice:

- Batch Normalization after every CONV layer
- Xavier initialization from He et al.
- SGD + Momentum (0.9)
- Learning rate: 0.1, divided by 10 when validation error plateaus
- Mini-batch size 256
- Weight decay of $1e-5$
- No dropout used

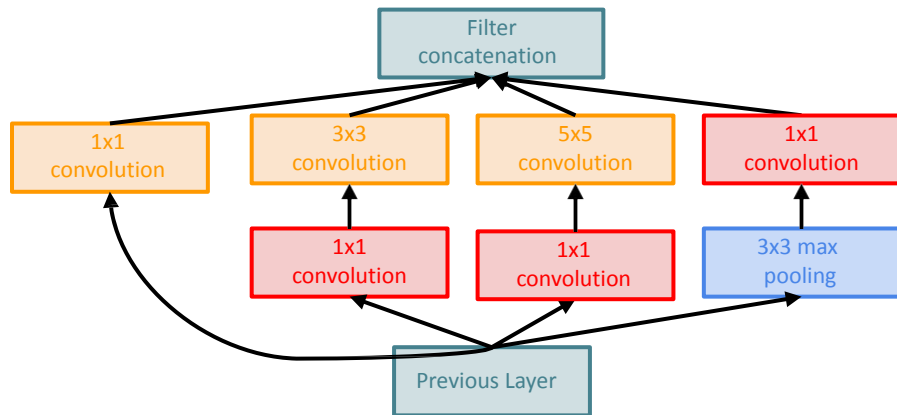
Inception Module - GoogLeNet

[Szegedy et al., 2014]

1x1 conv “bottleneck”
layers



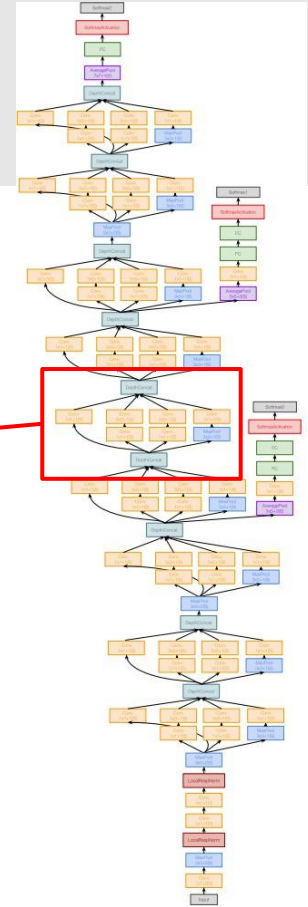
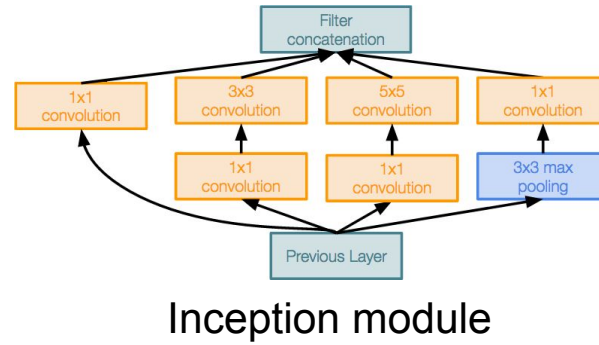
Naive Inception module



Inception module with dimension reduction

Inception Module - GoogLeNet

Stack Inception modules with dimension reduction on top of each other

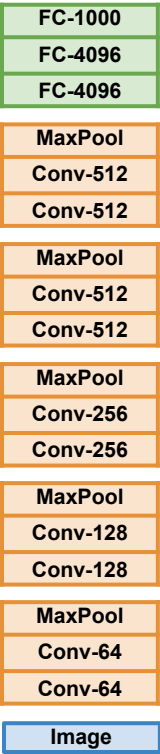


Transfer Learning

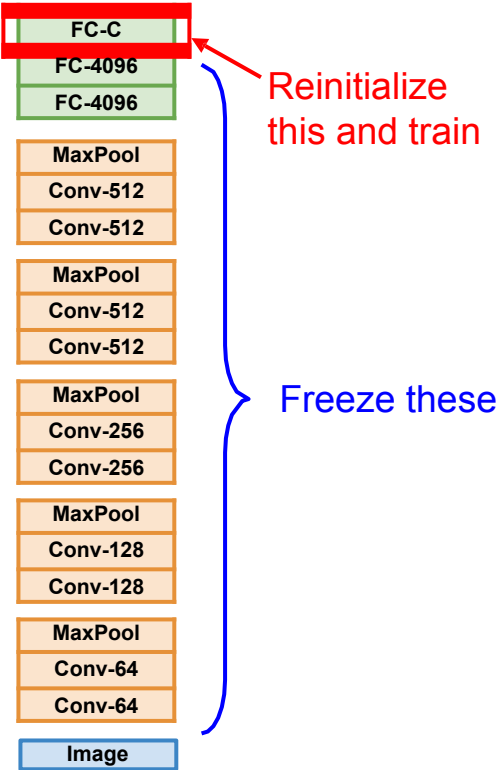
Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

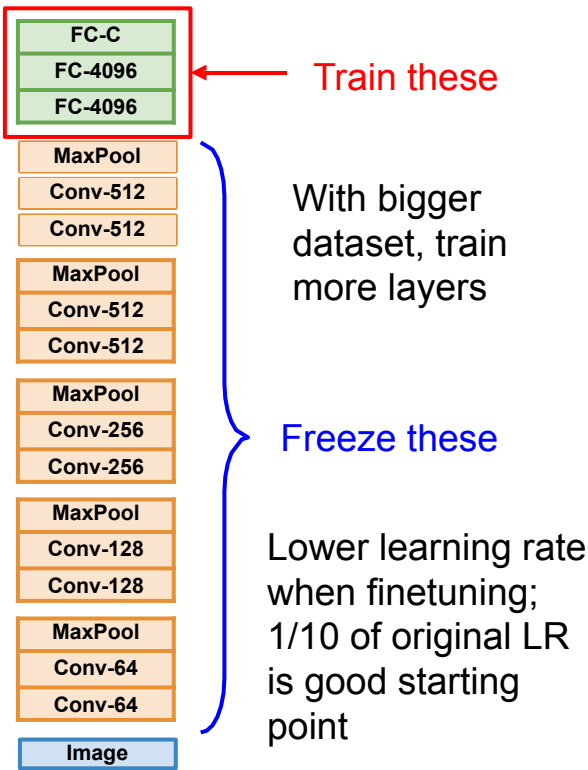
1. Train on Imagenet

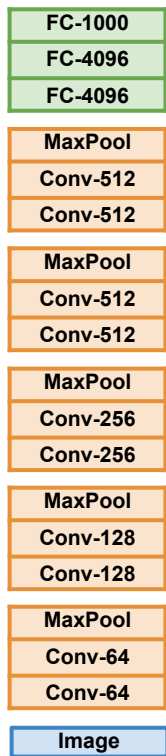


2. Small Dataset (C classes)



3. Bigger dataset





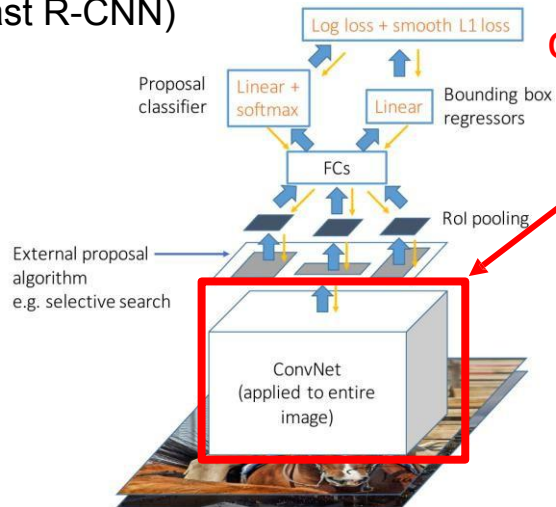
More specific

More generic

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	? Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers or start from scratch!

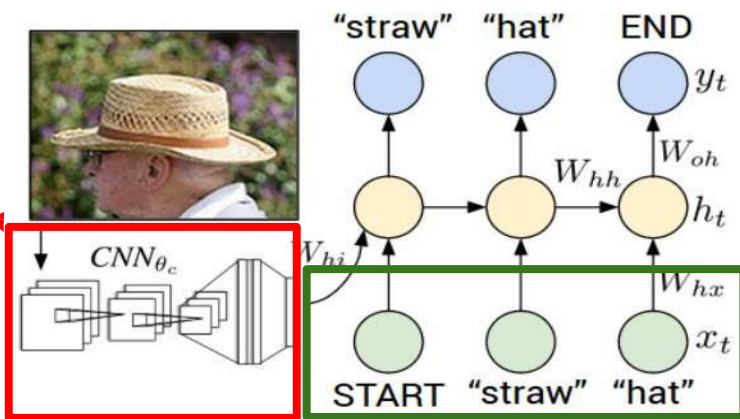
Transfer learning with CNNs is pervasive...

Object Detection (Fast R-CNN)



CNN pretrained
on ImageNet

Image Captioning: CNN + RNN



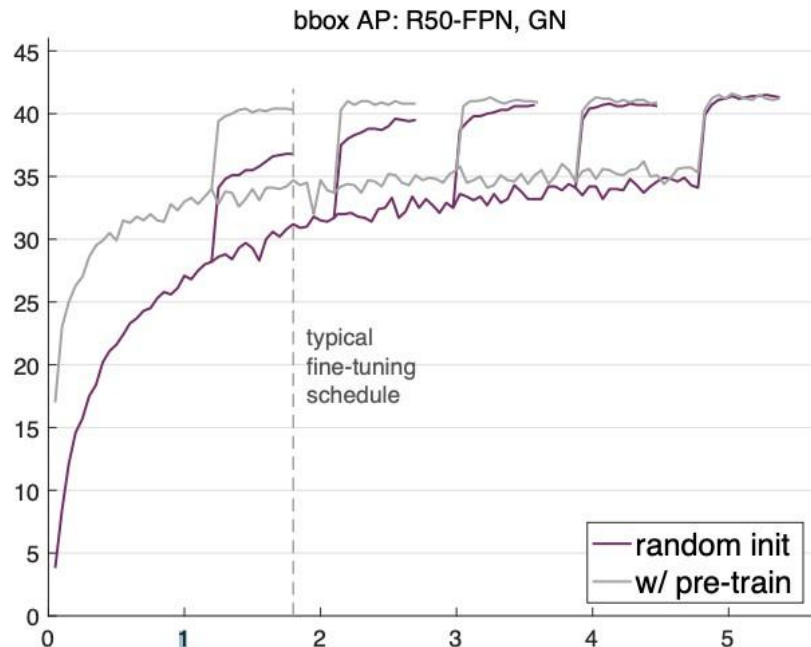
Word vectors pretrained
with word2vec

Girshick, "Fast R-CNN", ICCV 2015
Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for
Generating Image Descriptions", CVPR 2015
Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer learning with CNNs is pervasive...

But it might not always be necessary!



He et al, "Rethinking ImageNet Pre-training", ICCV 2019
Figure copyright Kaiming He, 2019. Reproduced with permission.

Training from scratch can work just as well as training from a pretrained ImageNet model for object detection

But it takes 2-3x as long to train.

They also find that collecting more data is better than finetuning on a related task

Efficient networks...



<https://openai.com/blog/ai-and-efficiency/>